

Hybrid Entity Resolution: Local Probabilistic Matching with Global Aggregation

A Study on Synthetic and Real-World Datasets

Steven Zhu

August 28, 2026

Abstract

This report details a hybrid methodology combining local probabilistic record linkage via SPLINK with global aggregation functions to resolve entity identity across a pair of data sources.

Experiments on synthetic data and public datasets demonstrate that the hybrid approach achieves strong precision-recall trade-offs when assumptions hold. At low cardinality, the lack of discriminatory factors reduce the performance of the model; we propose 1. A profile-based framework for comparison, and 2. A secondary SPLINK layer over aggregation scores.

Contents

1	Introduction	6
1.1	Problem Statement	6
1.2	Contributions	6
1.3	Report Structure	7
2	Theoretical Framework & Methodology	7
2.1	Foundations: The Fellegi-Sunter Probabilistic Model	7
2.1.1	Model Setup	7
2.1.2	Match Weight and Posterior Probability	8
2.1.3	Parameter Estimation	8
2.2	Two Frameworks for Global Entity Resolution	8
2.3	Framework 1: Score Aggregation (Local $\rightarrow$ Global)	10
2.3.1	Local Column Comparison Functions (Framework 1 Input)	10
2.3.2	Aggregation Functions	10
2.3.3	Extension: Using aggregation functions as second-level comparisons	11
2.4	Framework 2: Profile Aggregation (Behavioural Profile Matching)	12
2.4.1	Behavioural Profile Construction	12
2.4.2	Profile Matching: Set Similarity on Profiles	12
2.4.3	Applying Splink to Profile Vectors	13
2.4.4	Set Similarity Comparison Functions	13
2.4.5	Numerical Range Overlap	13
2.5	Blocking Strategy & Parameter Estimation Pipeline	14
2.5.1	Underlying Probabilities: m , u , and the Prior λ	14
2.5.2	Operational Blocking Stages and Mathematical Rationales	14
3	System Implementation & Tooling	16
3.1	Data Generation: Synthetic Corruption Pipeline	16
3.1.1	Configuration (YAML)	16
3.1.2	Noise Models	17
3.1.3	Latent Event IDs	17
3.2	Profile Aggregation Engine	17
3.3	Splink Configuration Overview	17
3.4	Blocking Rules Design	18
3.5	EM Training Schedule	18
4	Experimental Methodology	18
4.1	Datasets	18
4.2	Evaluation Protocol	19
4.2.1	Synthetic Data: Ground-Truth Available	19
4.2.2	Public Data: No Ground Truth	19
4.2.3	Metrics	19
4.3	Baselines and Ablations	19
4.4	Computational Environment	19
5	Experimental Results & Discussion	20
5.1	Experiment 1: base Synthetic Dataset - <code>splink_local_demo.ipynb</code>	20
5.1.1	Dataset Description	20
5.1.2	Blocking Rules Design	20
5.1.3	Comparison Levels	20
5.1.4	Local Pairwise Matching Results	20

5.1.5	Threshold Selection and Results	22
5.1.6	Local matching - Discussion	22
5.1.7	Blocking Efficiency	22
5.1.8	Local-to-Global Aggregation Analysis	22
5.1.9	Global Aggregation - Discussion	24
5.2	Experiment 2: Synthetic Dataset Variants	24
5.2.1	Variant Overview	24
5.2.2	Configuration	25
5.2.3	Results Summary	25
5.2.4	Intent	25
5.2.5	smallest Variant	25
5.2.6	Local-to-Global Aggregation Analysis	26
5.2.7	low_card Variant	28
5.2.8	Local-to-Global Aggregation Analysis	29
5.2.9	o50 Variant	31
5.2.10	Local-to-Global Aggregation Analysis	32
5.3	Experiment 3: Public Dataset Validation: MusicBrainz	34
5.3.1	Description	34
5.3.2	Configuration	34
5.3.3	Blocking Efficiency	35
5.3.4	Local-to-Global Aggregation Analysis	37
5.3.5	Discussion	38
5.4	Experiment 4: Behavioural Aggregation Experiments	38
5.4.1	Description	38
5.4.2	Configuration	39
5.4.3	Discussion	45
5.5	Experiment 5: Hierarchical 2-Layer Splink Approach	45
5.5.1	Description	45
5.5.2	Configuration	45
5.5.3	Results	45
5.5.4	Discussion	47
6	Edge Cases & Structural Extremities: Cardinality Breakdown	48
6.1	Failure Modes at Low Cardinality	48
6.1.1	Independence Assumption Violation	48
6.1.2	Combinatorial Explosion in Comparison Space	48
6.1.3	Sparse Parameter Estimation	48
6.1.4	Aggregation robustness	48
7	Discussion	48
7.1	On confidence scores and the Fellegi-Sunter model	48
7.2	Splink, and modern Fellegi-Sunter	50
7.3	Limitations	50
8	Future Work & Conclusion	50
8.1	Future Work	50
8.1.1	1. Automated Threshold Selection via Mixture Models	50
8.1.2	Further exploration into hierarchical probabilistic linkage	51
8.1.3	Active Learning & Label Minimisation	51
8.1.4	Multi-Source ($K > 2$) Global Resolution	51
8.1.5	Distributed SPLINK on Spark for Scale	51
8.1.6	Temporal Entity Resolution	52

8.1.7 Semantic Augmentation (LLM Hybrid)	52
8.2 Conclusion	52
A Summary of Entity Resolution Methodologies	53
B Mathematical Derivations	55
C Fellegi-Sunter Model Derivation	55
C.1 From Hypothesis Test to Match Weight	55
C.2 Posterior Match Probability	55
C.3 EM Algorithm Derivation	55
C.4 Blocked EM	56
D Aggregation Function Properties	56
E Set Similarity Metrics	56
E.1 Monge-Elkan	56
E.2 Soft Jaccard	56
E.3 Magnitude-Weighted Cosine	57
F Cardinality Breakdown Analysis	57
G Data Generation Implementation	57
H Main Entry Point: generate_data.py	57
I Configuration: data_generator/config.yaml	60
J Core Modules	61
J.1 data_generator/entitymanager.py	61
J.2 data_generator/pipeline.py	61
J.3 data_generator/generator.py	62
K Unified Soft String-Matching Metrics Framework	62
L Definitions and Notation	62
M The Four Soft Metrics	63
M.1 Monge-Elkan Similarity (Asymmetric)	63
M.2 Symmetric Soft Jaccard Similarity	63
M.3 Symmetric Soft Dice Coefficient	63
M.4 Soft Simpson Coefficient (Overlap Metric)	63
N Unified Binned Comparison Logic	63
O DuckDB SQL Implementations	63
O.1 Soft Jaccard	63
O.2 Monge-Elkan	64
O.3 Soft Dice	64
O.4 Soft Simpson	64
P Properties Summary	65
Q Usage Recommendations	65

R	Artist Matching Ground Truth and Precision/Recall Guarantees	65
R.1	Subset Precision is a Lower Bound on True Precision	65
R.2	Artist Matching Prior and Recall Estimation under Clustered Noise	66
R.2.1	Preservation of the True Match Prior (p)	66
R.2.2	Clustered Noise and Captured Predictive Inflation (q)	66

1 Introduction

1.1 Problem Statement

Entity resolution (ER)—the task of identifying records that refer to the same real-world entity across one or more data sources—is a foundational challenge in data integration. The problem arises from pervasive data quality issues: attribute ambiguity (e.g., ‘J. Smith’ vs. ‘John Smith’), data entry errors, missing values, temporal drift in attributes, and schema heterogeneity. Traditional deterministic approaches (exact matching, rule-based systems) are labour-intensive, brittle, and fail to generalise across diverse corruption patterns.

ER encompasses a wide spectrum of problem structures and inference methods: clustering-based, probabilistic models, supervised learning, etc. An annex summarising these methodologies is provided in [Appendix A](#).

The choice of method is dictated not by preference but by the *constraints* of the specific ER problem at hand. This report addresses ER under the following **explicit constraints**:

1. **Global entity resolution:** The goal is to resolve entities across two datasets (Source A and Source B) into a unified global view.
2. **Unsupervised learning:** No ground-truth labels are available for training; parameter estimation must be data-driven.
3. **Stable within-dataset identifiers:** Records in each source carry a LOCAL ENTITY identifier that is consistent for all records belonging to the same GLOBAL ENTITY *within that dataset*. Between datasets, the identifier schema differs ($DF1_*$ vs $DF2_*$), but the within-dataset stability assumption scopes us away from clustering/graph-based methods that would otherwise be necessary to discover local groups.
4. **Attribute types:** Matching signals derive from textual, numerical, and datetime columns only—scoping out LLM-based semantic matching and behavioural/network-based methods.
5. **Cost minimisation:** The objective is to minimise total resolution error (or equivalently, maximise F_1) under computational budget constraints imposed by blocking.

These constraints narrow the methodological space substantially: the FELLEGI-SUNTER probabilistic model with expectation-maximisation (EM) estimation, implemented via SPLINK, emerges as the natural (and arguably **only**) principled choice for matching.

1.2 Contributions

1. **Formalised Local-to-Global Framework** (Section 2): A clear separation between local probabilistic matching (FELLEGI-SUNTER via SPLINK) and global aggregation, with explicit assumptions and failure modes.
2. **Implementation Patterns for Unsupervised SPLINK** (Section 3): Blocking strategy design with recall bounds, custom comparison levels (exact, JARO-WINKLER, percentage-difference, temporal), EM training schedules, and evaluation without labels.
3. **Global Aggregation Taxonomy** (Section 2.3): Mathematical unification of naive sum, expected-score normalisation, noisy-OR, max-over-partitions, and vector-space aggregation.

4. **Experimental Validation** (Section 5): Systematic results on synthetic data and public datasets (MusicBrainz).
5. **Cardinality Extremities & Layer-2 Mitigation** (Section 6): Diagnosis of independence-assumption breakdown at low cardinality; proposal for alternative approaches.

1.3 Report Structure

This report alternates focus between *rationale* (why a design decision was made) and *practical implementation* (how it was realised in code).

- **Section 2:** Theoretical framework—FELLEGI-SUNTER foundations, SPLINK implementation, global aggregation framework, hierarchical probabilistic linkage.
- **Section 3:** System implementation—data generation, SPLINK configuration, blocking strategy.
- **Section 4:** Experimental methodology—datasets, metrics, evaluation protocol.
- **Section 5:** Experimental results—artificial baseline, public dataset validation, behavioural aggregation.
- **Section 6:** Edge cases—cardinality failure modes.
- **Section 7:** Discussion—confidence scores necessity, practical takeaways.
- **Section 8:** Future work and conclusion.
- **Annexes:** ER methodology summary, mathematical derivations

2 Theoretical Framework & Methodology

2.1 Foundations: The Fellegi-Sunter Probabilistic Model

Both frameworks build on the FELLEGI-SUNTER probabilistic record linkage model [Fellegi and Sunter \[1969\]](#), implemented via SPLINK [Linacre et al. \[2024\]](#). Full derivations are in Appendix B.

2.1.1 Model Setup

For a record pair (r_1, r_2) , we observe a **comparison vector** $\gamma = (\gamma_1, \dots, \gamma_K)$ where each γ_k is the comparison outcome for field k (e.g., exact match, Jaro-Winkler band, percentage-difference bucket). The linkage task is a hypothesis test between match (H_M) and non-match (H_U).

The model assumes **conditional independence** of field comparisons given match status:

$$\mathbb{P}(\gamma \mid M) = \prod_{k=1}^K m_k(\gamma_k), \quad \mathbb{P}(\gamma \mid U) = \prod_{k=1}^K u_k(\gamma_k) \quad (1)$$

where $m_k(\gamma) = \mathbb{P}(\gamma_k = \gamma \mid M)$ is the *m-probability* (probability of observing comparison value γ on field k among true matches) and $u_k(\gamma) = \mathbb{P}(\gamma_k = \gamma \mid U)$ is the *u-probability* (same for non-matches). The prior match probability in the comparison space is $\pi = \mathbb{P}(M)$.

2.1.2 Match Weight and Posterior Probability

Remark: The match weights and match probabilities have a unique one-to-one mapping; we can convert between them seamlessly given the match prior π . The weights are additive and intuitive for feature debugging, while the final probabilities are calibrated for operational decision-making. [Winkler, 1993]

The likelihood ratio is $\Lambda(\gamma) = \mathbb{P}(\gamma | M) / \mathbb{P}(\gamma | U)$. Under conditional independence, the **match weight** (log-likelihood ratio, base 2) decomposes additively:

$$w(\gamma) = \log_2 \Lambda(\gamma) = \sum_{k=1}^K \log_2 \frac{m_k(\gamma_k)}{u_k(\gamma_k)}. \quad (2)$$

Each field contributes a weight $w_k(\gamma_k)$ that depends on the observed comparison level—exact match yields large positive weight ($m \gg u$), mismatch yields large negative weight ($u \gg m$), partial match yields an intermediate weight, enabling fuzzy matching.

By Bayes’ theorem, the posterior match probability is:

$$\mathbb{P}(M | \gamma) = \frac{\pi \cdot 2^{w(\gamma)}}{\pi \cdot 2^{w(\gamma)} + (1 - \pi)}. \quad (3)$$

Equivalently, $w = \log_2 \left(\frac{\mathbb{P}(M|\gamma)}{1-\mathbb{P}(M|\gamma)} \cdot \frac{1-\pi}{\pi} \right)$.

2.1.3 Parameter Estimation

Remark: The EM algorithm learns the unique discriminatory value of each attribute based on how frequently features match or differ across your data, allowing it to automatically figure out which columns deserve a higher match weight.

The parameters $\{m_k, u_k, \pi\}$ are estimated via the **Expectation-Maximisation (EM) algorithm** Dempster et al. [1977] on a blocked comparison space. SPLINK initialises π using a strict blocking rule (high precision) and a target recall R , then iterates E-step (compute posteriors $\tau_i = \mathbb{P}(M | \gamma_i)$) and M-step (re-estimate m_k, u_k, π weighted by τ_i) until convergence. Full details are in Appendix C.3.

2.2 Two Frameworks for Global Entity Resolution

We study two distinct frameworks for resolving GLOBAL ENTITIES under the assumption of **stable within-dataset identifiers**: records sharing a `unique_id` belong to the same GLOBAL ENTITY. The frameworks differ in *where* and *how* the local-to-global transition occurs.

Framework 1: Score Aggregation (Local → Global) Perform local probabilistic record linkage (FELLEGI-SUNTER via SPLINK) on individual record pairs, producing a matrix of match weights/probabilities per GLOBAL ENTITY pair. Aggregate this matrix to a single global score using scalar functions (Sum, Mean, Noisy-OR, Max) or vector-space aggregation.

Framework 2: Profile Aggregation (Behavioural Profile Matching) We compress each GLOBAL ENTITY’s records into a fixed-dimensional **behavioural profile** (a latent feature vector/set summarising its multi-record behaviour). Using set-similarity functions for comparisons, perform probabilistic linkage on these profiles (FELLEGI-SUNTER via SPLINK) to produce a scalar global match probabilities per record pair.

Table 1 illustrates the architectural distinction.



Figure 1: Framework 1 (Score Aggregation). Records are compared pairwise by Fellegi-Sunter, producing per-field comparison levels γ_k and a matrix of match probabilities P per global entity pair. Scalar aggregation (e.g., Noisy-OR) yields a global score. For (e_1, e_1) , multiple corroborating buy-buy pairs drive the score ≈ 1 ; for (e_2, e_2) , a single buy-sell mismatch yields a low score.

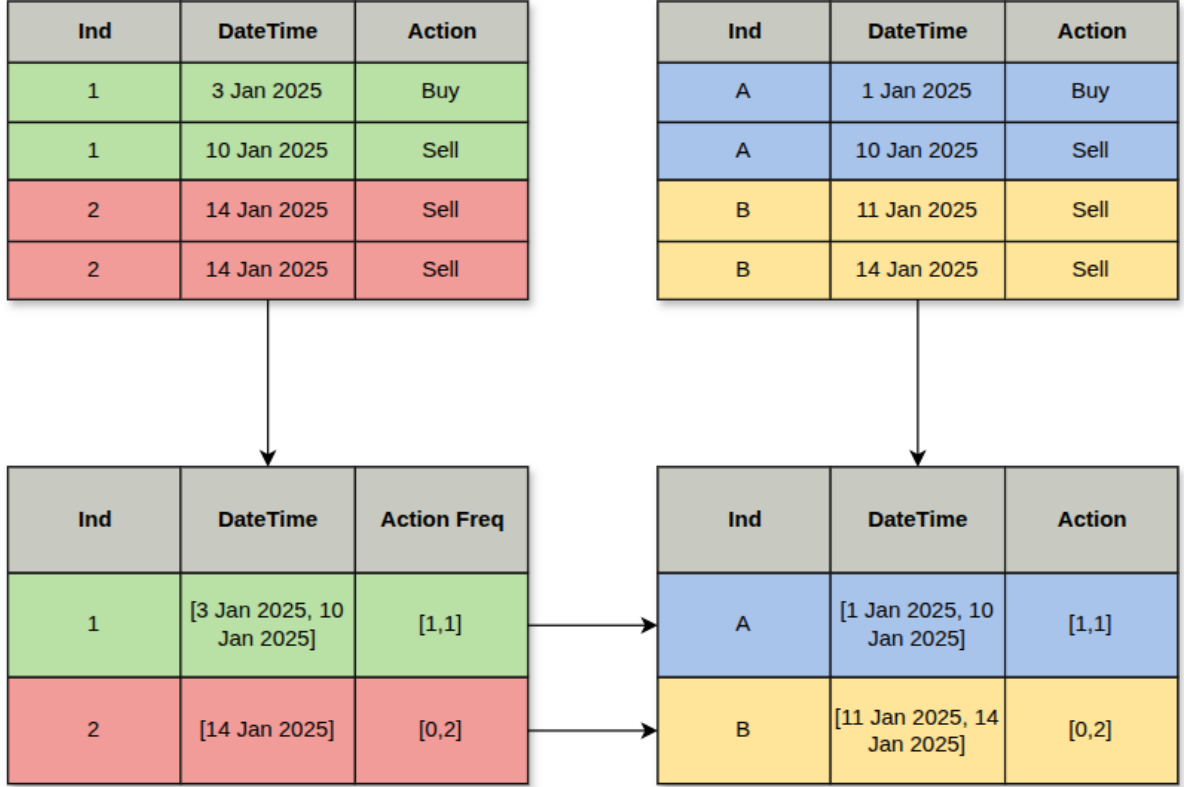


Figure 2: Framework 2 (Profile Aggregation). Records are first aggregated per entity into profiles (sets of actions, datetime ranges, frequency vectors). Set similarity metrics (Soft Jaccard, Monge-Elkan, Simpson, Range Overlap) compute per-dimension similarities $\mathbf{x} = (x_1, \dots, x_D)$. A Fellegi-Sunter model then learns discriminative weights and outputs calibrated posteriors. For (e_1, e_1) , overlapping actions and close datetimes yield high similarity; for (e_2, e_2) , disjoint actions yield near-zero similarity.

Table 1: Framework Comparison

Aspect	Score Aggregation	Profile Aggregation
Local-to-Global Transition	<i>After</i> matching (on scores)	<i>Before</i> matching (on records)
Core Operation	$\mathcal{A}(\{w_{ij}\}_{i=1..m, j=1..n})$	$\text{sim}(\text{Agg}(\{r_i\}), \text{Agg}(\{r'_j\}))$
Theoretical Grounding	FELLEGI-SUNTER on local pairs	FELLEGI-SUNTER on profiles
Interpretability	Per-field weights γ_k	Profile feature contributions

2.3 Framework 1: Score Aggregation (Local $\rightarrow$ Global)

In this framework, the local layer produces a matrix of match probabilities $P \in [0, 1]^{m \times n}$ (or weights $W \in \mathbb{R}^{m \times n}$) for each GLOBAL ENTITY pair (e_A, e_B) with $m = |R_A|$, $n = |R_B|$. The global score is $s(e_A, e_B) = \mathcal{A}(P)$.

2.3.1 Local Column Comparison Functions (Framework 1 Input)

The local FELLEGI-SUNTER layer requires comparison functions $\gamma_k(r_1, r_2)$ for each field k . These produce the discrete, binned *gamma values* that feed into the m/u probability model. We use type-specific comparison levels implemented as `Splink CustomComparison`:

- **Categorical (Exact Match):** $\gamma_{\text{cat}}(r_1, r_2) = \mathbb{I}[r_{1,\text{cat}} = r_{2,\text{cat}}]$.
- **Numerical (Percentage Difference):** $\gamma_{\text{num}}(r_1, r_2) = \frac{|r_{1,\text{num}} - r_{2,\text{num}}|}{\max(r_{1,\text{num}}, r_{2,\text{num}})}$.
- **Datetime (Days Apart):** $\gamma_{\text{dt}}(r_1, r_2) = |r_{1,\text{dt}} - r_{2,\text{dt}}|_{\text{days}}$.
- **Text (Jaro-Winkler):** $\gamma_{\text{text}}(r_1, r_2) = \text{JW}(r_{1,\text{text}}, r_{2,\text{text}})$.
- **Set-Valued (Soft Jaccard / Monge-Elkan / Simpson):** As defined in Section 2.4.4.

For each comparison, we deterministically bin the resulting similarity scores into distinct comparison levels (γ_k). The underlying conditional probabilities— $m_k(\gamma)$ and $u_k(\gamma)$ —are learned dynamically during the EM optimization loop. Introducing more comparison levels captures finer similarity distinctions, but it scales the parameter space and risks model overfitting due to data sparsity within narrower bins.

The conditional probabilities are then used to compute the match weight $w_k(\gamma_k)$ and posterior match probability $\mathbb{P}(M \mid \gamma)$ for each local record pair. The matrix of match probabilities, P , are then aggregated as the subsequent section describes.

2.3.2 Aggregation Functions

Given the matrix P of local match probabilities, the global score is $s(e_A, e_B) = \mathcal{A}(P)$ with $\mathcal{A}$ from the following taxonomy:

- **Noisy-OR:** $s_{\text{noisy-or}} = 1 - \prod_{i,j} (1 - p_{ij})$. Probability that *at least one* local pair is a true match; saturates quickly with many pairs.
- **Max-Over-Partitions:** $s_{\text{max}} = \max_{i,j} p_{ij}$. Takes strongest local evidence; robust to false positives but discards corroborating evidence.
- **Vector-Space Aggregation:** Compute a score vector per entity (e.g., per-field mean weights) and compare via cosine/Jaccard. Retains more structure but still derives from local pairwise scores.

- **Match Density:** $D_{AB} = \frac{\sum_{i \in A, j \in B} p_{ij}}{|A| \cdot |B|}$, representing the average match probability across all record pairs between entities A and B .
- **Weighted Jaccard:** $J_w(A, B) = \frac{\sum p_{ij}}{|A| + |B| - \sum p_{ij}}$, a generalization of the Jaccard index where record-pair match probabilities weight the intersection.
- **Weighted Overlap:** $O_w(A, B) = \frac{\sum p_{ij}}{\min(|A|, |B|)}$, measuring the proportion of the smaller entity's records that participate in matching pairs.
- **Score-Probability Expected Ratio:** $R_{AB} = \log \frac{\sum p_{ij} + \varepsilon}{|A||B|u + \varepsilon}$, where u is the base match probability (prior) and ε a small smoothing term. Contrasts observed against expected matches under independence.

Theoretical Properties (proofs in Appendix D):

- Noisy-OR is monotone in each p_{ij} and in m, n (adding pairs never decreases the score).
- Max is invariant to dominated pairs.
- Match density assumes rank-1 structure $p_{ij} = a_i b_j$ for cardinality invariance.
- All scalar aggregations assume exchangeability of local pairs—violated under cardinality imbalance.

2.3.3 Extension: Using aggregation functions as second-level comparisons

As an alternative to manually selecting a single baseline aggregation function from the choices outlined above, this report further explores an approach involving framing these very functions as inputs to a *secondary, hierarchical probabilistic linkage stage*.

Instead of arbitrarily selecting between the structural trade-offs of various aggregation functions, we can treat the scalar outputs of these four functions as an entirely new set of group-level **comparison vectors**.

An analogous approach: In a standard Fellegi-Sunter run, we map raw data fields into discrete comparison levels (γ_k) and let the EM algorithm dynamically learn their conditional probabilities (m and u).

Analogously, we can continuous-bin or discretise the scores from aggregation functions into group-level structural traits. Passing this vector of structural traits into a second-stage EM loop yields distinct weights for each level, effectively quantifying the empirical discriminatory power of each mathematical lens based on the observed distribution of global entities versus accidental data noise.

We consider two major benefits to utilising this design, despite violating Conditional Independence:

Beneficial Double-Counting (Probabilistic Voting): In traditional record linkage, violating conditional independence in attributes can aggressively drive scores to the extremes (0.0 or 1.0). Treating matches on two correlated fields as miraculous evidence inflates match weights. Matches on postal code and addresses should not be treated as independent evidence.

However, a true, structurally sound cluster will simultaneously clear the mathematical bars for multiple independent traits. Matches on multiple aggregation functions *should be much stronger evidence* that two entities are the same. The resulting amplified weight acts as a *probabilistic voting mechanism*, handing true matches a massive cumulative bonus while rapidly penalising noisy, accidental links that fail to satisfy all lenses simultaneously.

Prioritisation of Ranking Over Absolute Calibration: For operational notes, the primary goal of the engine is discrimination power rather than exact statistical calibration. While the CI violation uncalibrates the absolute probabilities into a highly bimodal distribution, it generally preserves the overall **ranking order** of the data. By widening the geometric gap between legitimate entity structures and chaotic random matches, this approach would establish a clean, unambiguous operational threshold boundary.

Implementation Note: This hierarchical approach remains strictly as a secondary option. If an ideal, clear aggregation function is available, it should be used instead to maintain strict mathematical independence.

2.4 Framework 2: Profile Aggregation (Behavioural Profile Matching)

This framework shifts the local-to-global transition *upstream*: it **aggregates records into profiles first**, then matches profiles with the FELLEGISUNTER model.

2.4.1 Behavioural Profile Construction

For each GLOBAL ENTITY e with records $R(e) = \{r_1, \dots, r_k\}$, a **profile vector** $\mathbf{v}_e$ is constructed via an **Aggregator**:

$$\mathbf{v}_e = \text{Agg}(\{\phi(r) : r \in R(e)\}) \quad (4)$$

where $\phi(r)$ extracts features from a single record, and Agg is a per-feature aggregation operator. The **aggregation** package (`em/aggregation/`) implements this with a **ProcessorRegistry** mapping feature types to **ColumnProcessor** subclasses:

- **Numerical** $\rightarrow$ moments: {mean, min, max} for range overlap comparison.
- **Set-valued (Categorical and DateTime)** $\rightarrow$ union of tokens across records.
- **Frequency Vectors (Categorical and DateTime)** $\rightarrow$ frequency vectors for Soft Jaccard / Monge-Elkan / Simpson / Mag-Cosine. Can quickly explode in dimensionality. [Winkler, 1989]

The output is a fixed-dimensional profile per entity.

2.4.2 Profile Matching: Set Similarity on Profiles

Given profiles $\mathbf{v}_A, \mathbf{v}_B$ for entities e_A, e_B , per-dimension similarities form a feature vector:

$$x_d = \text{sim}_d(v_{A,d}, v_{B,d}), \quad d = 1 \dots D \quad (5)$$

$$\mathbf{x} = (x_1, \dots, x_D) \quad (6)$$

where sim_d are the soft set metrics (Soft Jaccard, Monge-Elkan, Simpson, Mag-Cosine, Range Overlap) applied per profile dimension d .

2.4.3 Applying Splink to Profile Vectors

A FELLEGI-SUNTER model is trained on these global-pair features:

- **Input:** DataFrame with one row per global entity pair, columns $x_1 \dots x_D$.
- **Blocking:** Use a high-recall profile similarity (e.g., Soft Jaccard ≥ 0.3) as blocking key.
- **EM Estimation:** Same procedure; each “record” is now a global entity pair.
- **Output:** Calibrated global match probability $\mathbb{P}(M \mid \mathbf{x})$.

2.4.4 Set Similarity Comparison Functions

For set-valued fields (tokenised titles, year lists, categorical number lists) the set similarity metrics are fundamentally **intersection-over-union variants**: compute a “soft intersection” by summing continuous match scores instead of exact token matches, then normalise by various denominator forms (union, sum, min). We implement four metrics in DuckDB SQL (Appendix K):

Let L and R be token lists. Define the directional best-match sums:

$$S_{L \rightarrow R} = \sum_{x \in L} \max_{y \in R} \text{JW}(x, y), \quad (7)$$

$$S_{R \rightarrow L} = \sum_{y \in R} \max_{x \in L} \text{JW}(x, y). \quad (8)$$

The “soft intersection” is $S_{L \rightarrow R} + S_{R \rightarrow L}$, replacing exact $|\text{tokens}_L \cap \text{tokens}_R|$.

Monge-Elkan (Asymmetric) $\text{ME}(L, R) = \frac{S_{L \rightarrow R}}{|L|}$. Normalises soft intersection by query size $|L|$. Query-to-reference average best-match.

Soft Jaccard (Symmetric) $\text{SJ}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{2 \cdot |L \cup R|_{\text{distinct}}}$. Normalises by (distinct) union size; penalises extra tokens on either side.

Simpson (Symmetric) $\text{SI}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{2 \cdot \min(|L|, |R|)}$. Normalises by twice the size of the smaller set.

Mag-Cosine (Symmetric) $\text{MC}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{\sqrt{(S_{L \rightarrow L} + S_{R \rightarrow R}) \cdot (S_{L \rightarrow R} + S_{R \rightarrow L})}}$. Normalises by the geometric mean of the self-similarities.

For numerical fields, we use the range overlap comparison. Given two intervals $[a_1, a_2]$ and $[b_1, b_2]$, the range overlap is defined as:

$$\text{RO}([a_1, a_2], [b_1, b_2]) = \frac{\max(0, \min(a_2, b_2) - \max(a_1, b_1))}{\max(a_2, b_2) - \min(a_1, b_1)} \quad (9)$$

This is the Jaccard index for intervals, measuring the overlap relative to the combined span.

2.4.5 Numerical Range Overlap

For numerical profiles (mean/min/max from Aggregator), we use the range overlap coefficient:

$$\text{Overlap}(A, B) = \frac{\max(0, \min(A_{\max}, B_{\max}) - \max(A_{\min}, B_{\min}))}{\max(A_{\max}, B_{\max}) - \min(A_{\min}, B_{\min})} \quad (10)$$

measuring the fraction of the bounding box that overlaps.

2.5 Blocking Strategy & Parameter Estimation Pipeline

Remark: We utilise blocking for both frameworks—critical for reducing the $\mathcal{O}(|A| \times |B|)$ comparison space. In Score Aggregation, blocking applies to *local record pairs*; in Profile Aggregation, it applies to *global entity pairs* using profile features as keys. We discuss the core rationale for specific choices governing blocking across SPLINK’s primary operational stages.

2.5.1 Underlying Probabilities: m , u , and the Prior λ

From 2.1, we re-establish the fundamental parameters governing the FELLEGI-SUNTER model for a comparison field i :

- **m-probability** (m_i): $\mathbb{P}(\text{field } i \text{ matches} \mid M)$ — the probability that a field agrees given that the record pair represents a true match (M). This parameter quantifies data entry error rates.
- **u-probability** (u_i): $\mathbb{P}(\text{field } i \text{ matches} \mid U)$ — the probability that a field agrees purely by random chance given that the record pair is a non-match (U). This reflects accidental data coincidences.
- **Prior** (λ): $\mathbb{P}(M)$ — the unconditional probability that two record pairs chosen completely at random from the entire population space represent a true match.

2.5.2 Operational Blocking Stages and Mathematical Rationales

Stage 1: Estimating the Prior (`estimate_probability_two_random_records_match`) Estimating λ natively within an open cross-product or a loose block introduces massive mathematical distortion. In two datasets of size N and M , the total comparison space of all possible pairs is rectangular:

$$P_{N,M} = N \times M \quad (11)$$

If the true match count is M_{true} , the true prior probability of a match is:

$$\lambda = \frac{M_{\text{true}}}{P_{N,M}} \quad (12)$$

, which is often extremely small.

Running Expectation-Maximisation (EM) directly on this space causes the true match signal to be mathematically drowned out by non-matches. Conversely, running EM inside a loose block yields an artificially inflated local prior ($\pi_{\text{block}} \gg \lambda$). This systematic overestimation propagates through the Fellegi-Sunter equations, inflating the final match weights and causing massive false-positive rates.

To resolve this "blocking bias," Splink decouples λ from EM by utilizing **highly conservative, strict deterministic blocking rules** (B_{det}) where the user can safely assume a near-zero false-positive rate (e.g., exact match on `first_name` $\cap$ `surname` $\cap$ `dob`).

Let N_{det} be the exact number of candidate pairs generated by B_{det} . Assuming these deterministic pairs represent a robust, clean empirical lower bound of true matches ($N_{\text{det}} \approx M_{\text{true}}$), the prior

is directly calculated via a closed-form combinatorial ratio without relying on latent variable iteration:

$$\lambda \approx \frac{N_{\text{det}}}{P_{N,M}} = \frac{2 \cdot N_{\text{det}}}{N(N-1)} \quad (13)$$

Stage 2: Estimating m -probabilities via Expectation-Maximisation Once λ and u_i are initialized, this stage relies on **strict, single-field or dual-field blocking rules** run sequentially (e.g., Run A blocks on `first_name`; Run B blocks on `dob`). The mathematical choices here are governed by two strict constraints:

1. **Information Density and EM Convergence:** Extreme class imbalance destabilizes the likelihood surface of the EM algorithm, leading to poor local minima or failure to converge. Tight blocking rules compress the denominator of the posterior probability, filtering out obvious non-matches. This yields a dense local mixture where the local block prior sits in an optimal training regime.
2. **The Conditional Identifiability Constraint:** If a model blocks on field i (e.g., requiring `l.first_name = r.first_name`), the observed comparison value for field i is identically 1 for every single pair in that subpopulation. Consequently, there is zero variance in that column, making m_i mathematically unidentifiable within that specific optimization run. Splink handles this by cycling blocking columns: Run A fixes field i to estimate $m_{\setminus i}$, while Run B fixes field j to estimate m_i .

The EM algorithm treats match status $z \in \{0, 1\}$ as a latent variable. For a pair k within the block characterized by comparison vector $\mathbf{x}_k$:

$$\text{E-step: } \gamma_k = \frac{\pi_{\text{block}} L_M}{\pi_{\text{block}} L_M + (1 - \pi_{\text{block}}) L_U} \quad (14)$$

$$\text{M-step: } m_i^{\text{new}} = \frac{\sum_k \gamma_k x_{ki}}{\sum_k \gamma_k}, \quad \pi_{\text{block}}^{\text{new}} = \frac{1}{B} \sum_k \gamma_k \quad (15)$$

$$\text{where: } L_M = \prod_i m_i^{x_{ki}} (1 - m_i)^{1-x_{ki}}, \quad L_U = \prod_i u_i^{x_{ki}} (1 - u_i)^{1-x_{ki}} \quad (16)$$

where B is the number of pairs in the block, L_M and L_U terms are match likelihoods.

Stage 3: Prediction and Inference (`linker.predict()`) During final inference, the mathematical objective flips completely from parameter optimization to **maximizing the global recall bound**. Splink requires an array of **multiple, separate, overlapping blocking rules** evaluated as an inclusive disjunction ($\bigvee B_m$).

1. **Typo Redundancy via Disjunctive Paths:** True matches often can, and will, fail any single strict blocking rule. If a true match contains a typo in the surname, it will violate B_1 (`surname` equality) but will be captured by B_2 (`dob` $\cap$ `postcode` equality). The total recall of the prediction space is bounded by the union of the rules:

$$\text{Recall} \left(\bigcup_{m=1}^M B_m \right) \geq \text{Recall}(B_m) \quad \forall m \quad (17)$$

2. **Computational Complexity and Join Optimization:** A single complex SQL query with internal `OR` statements forces database engines into a slow nested-loop join scaling at $\mathcal{O}(N^2)$. Splink circumvents this by splitting the logic into independent equi-join rules.

This allows the engine to utilize fast linear-time **Hash Joins** scaling at $\mathcal{O}(N)$ lookup complexity. The separate outputs are then unified to compute the final Fellegi-Sunter log-odds weight:

$$W = \log_2 \left(\frac{\lambda}{1 - \lambda} \right) + \sum_i \log_2 \left(\frac{m_i}{u_i} \right) \quad (18)$$

3 System Implementation & Tooling

3.1 Data Generation: Synthetic Corruption Pipeline

The synthetic data generator (`generate_data.py`, `data_generator/`) creates two datasets A and B with known GLOBAL ENTITY overlap. The pipeline (Figure 3):

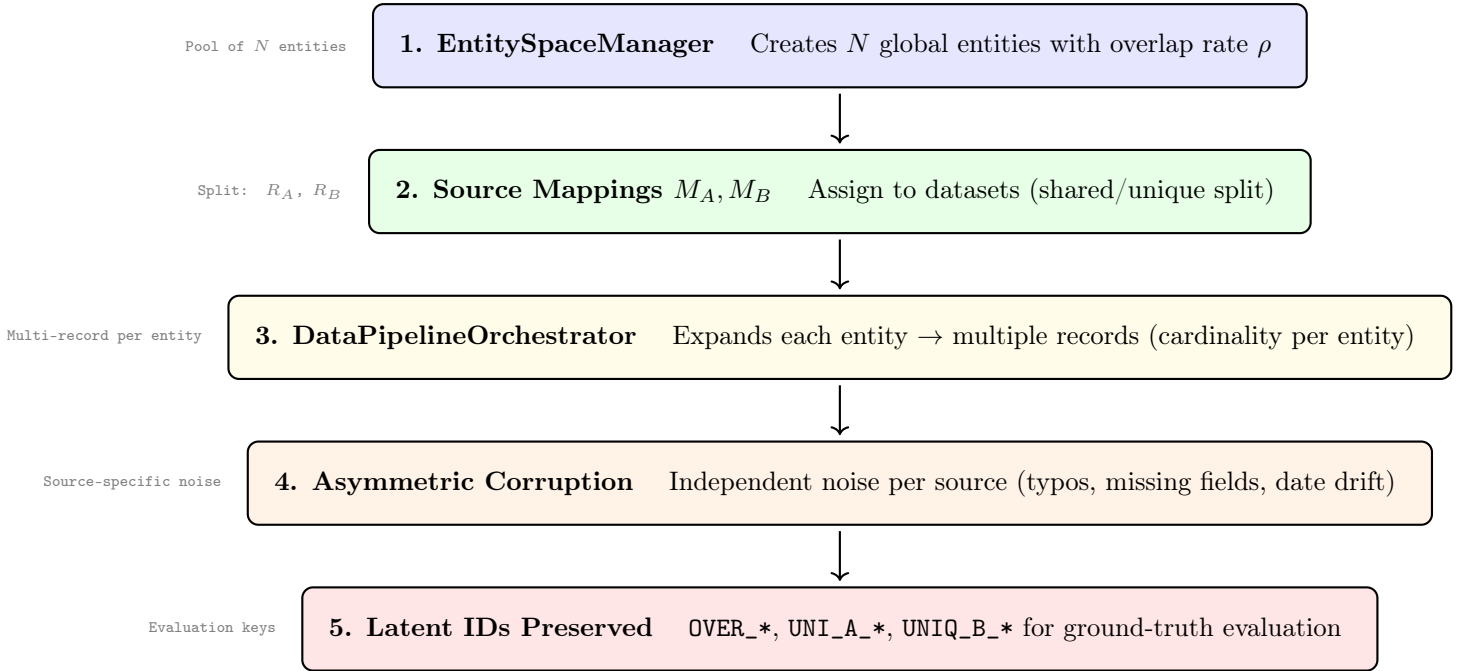


Figure 3: Synthetic Data Generation Pipeline. Vertical stages show the transformation from entity pool $\rightarrow$ split datasets $\rightarrow$ multi-record expansion $\rightarrow$ independent corruption $\rightarrow$ ground-truth keys. Each stage is a separable, testable component in the codebase.

The pipeline was designed to easily support customisations at various levels (config, data type, data distributions).

3.1.1 Configuration (YAML)

Key parameters (`data_generator/config.yaml`):

- **dataset_size:** Total global entities N .
- **overlap_percentage:** Fraction ρ of entities present in both datasets.
- **source_a_rows_per_entity, source_b_rows_per_entity:** Multiplicity (cardinality) per entity per source.
- **features:** List of $\{\text{name, type, noise_params, missing_rate}\}$. Types: **categorical** (classes, missing), **numerical** (Gaussian noise σ), **datetime** (Gaussian hours, missing).

- `overlap_dropout_frac`: Asymmetric dropout on overlapping portion of B.

3.1.2 Noise Models

- **Categorical**: Uniform random generation out of `num_classes`, missing with `missing_rate`.
- **Numerical**: True value + $\mathcal{N}(0, \sigma^2)$; missing with `missing_rate`.
- **Datetime**: True timestamp + $\mathcal{N}(0, \sigma_{\text{hours}}^2)$ hours; missing with `missing_rate`.

3.1.3 Latent Event IDs

Every generated row receives a `latent_event_id`:

- `OVER_k`: Overlapping entities (same k in A and B $\rightarrow$ true match).
- `UNI_A_k`: Unique to A.
- `UNIQ_B_k`: Unique to B.

These enable ground-truth evaluation without manual labelling.

3.2 Profile Aggregation Engine

The profile aggregation engine transforms raw record tables into entity-level profile vectors. A simplified horizontal flow diagram illustrates the transformation:

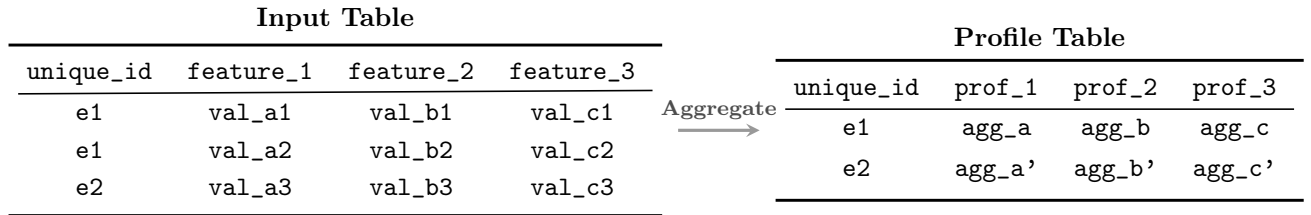


Figure 4: Horizontal flow of the profile aggregation engine: raw records are grouped by entity identifier and transformed into fixed-dimensional profile vectors.

The full implementation resides in the `em/aggregation/` package. It uses a `ProcessorRegistry` to map feature types to `ColumnProcessor` subclasses, and an `Aggregator` that coordinates multiple processors, builds vocabularies, applies aggregation functions, and runs post-processors.

3.3 Splink Configuration Overview

The Entity Matching pipeline uses the Splink library for probabilistic record linkage. All synthetic datasets share identical comparison logic; only dataset paths and resource parameters vary. Notebooks with real datasets reuse the same structure with domain-specific field mappings.

Supported comparison functions include:

- Basic Splink levels: `NullLevel`, `ExactMatchLevel`, `ElseLevel`.
- Binned levels: `PercentageDifferenceLevel`, `WithinNDaysLevel`, `JaroWinklerLevel`.
- Custom comparisons via SQL expressions (e.g., for set similarities, range overlap).

The core workflow follows the Splink transaction demo pattern:

1. Define comparison levels for each field.

2. Specify blocking rules as SQL predicates.
3. Train the model via expectation-maximisation (EM) using a fixed schedule of blocking rules.
4. Generate predictions and evaluate against known latent IDs.

Full code listings are in the respective notebooks mentioned by the report.

3.4 Blocking Rules Design

Blocking rules are supplied as an array of SQL predicates to the argument `blocking_rules_to_generate_predictions`. Each rule targets a different matching signal (e.g., categorical equality, numerical proximity, temporal proximity) and their union bounds recall while reducing the comparison space. Example predicates (for synthetic data) are of the form:

- `l.categorical = r.categorical OR l.categorical IS NULL OR r.categorical IS NULL`
- `strftime(l.datetime_noisy, '%Y%m') = strftime(r.datetime_noisy, '%Y%m') OR l.datetime_noisy IS NULL OR r.datetime_noisy IS NULL`
- `(ABS(l.numerical_noisy - r.numerical_noisy) / NULLIF(CASE WHEN l.numerical_noisy > r.numerical_noisy THEN l.numerical_noisy ELSE r.numerical_noisy END, 0)) < 0.25 OR l.numerical_noisy IS NULL OR r.numerical_noisy IS NULL`

Full code listings are in the respective notebooks mentioned by the report.

3.5 EM Training Schedule

Parameter estimation proceeds via a fixed sequence of expectation-maximisation steps, each using a different blocking rule to estimate m-probabilities for the remaining comparisons. The schedule begins with estimating the global prior from strict rules, then estimates u-probabilities via random sampling, followed by EM rounds that alternate blocking columns to identify all m-probabilities.

Full code listings are in the respective notebooks mentioned by the report.

4 Experimental Methodology

4.1 Datasets

The naming for the datasets are completely arbitrary, and hold little meaning towards the actual config and structure of the datasets.

Table 2: Datasets Used in Experiments

Name	Type	Entities	Overlap	Cardinality	Source
base	Synthetic	50	30%	70/80	generate_data.py
smallest	Synthetic	50	10%	35/45	generate_data.py
low_card	Synthetic	100	10%	18/12	generate_data.py
o50	Synthetic	50	50%	30/35	generate_data.py
MusicBrainz	Real	3,900 / 3,800	27%	Multi / Multi	Public (MusicBrainz)

4.2 Evaluation Protocol

4.2.1 Synthetic Data: Ground-Truth Available

Since `latent_event_id` is preserved, we construct a clerical labels table:

1. Merge predictions with ground truth on `latent_event_id_l == latent_event_id_r`.
2. Compute TP, FP, FN, TN at each probability threshold.
3. Derive Precision, Recall, F_1 , Accuracy.
4. Select optimal threshold via maximum F_1 .

4.2.2 Public Data: No Ground Truth

For MusicBrainz, we rely on:

- **Blocking recall bounds:** Theoretical upper bound on recall from blocking rule coverage.
- **Precision/Recall estimation:** Using the SPLINK accuracy analysis with a small manual clerical sample (if available) or synthetic proxy.
- **Stability checks:** Consistency of match weights across EM rounds; separation of match/non-match weight distributions.

4.2.3 Metrics

$$\text{Precision} = \frac{TP}{TP + FP} \quad (19)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (20)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (21)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} \quad (22)$$

For global aggregation experiments, we also report **global entity-level** metrics after aggregation (matching on `global_entity_id`).

4.3 Baselines and Ablations

Normalised Aggregation Mean probability per global pair; threshold.

Noisy-OR Aggregation Probabilistic union; threshold.

Max Aggregation Max probability per global pair; threshold.

Vector-Space Aggregation Profile vectors, second SPLINK layer (Section 2.4).

Data Config Variants Varying data structure.

4.4 Computational Environment

- DuckDB in-memory with 24GB memory limit, spill to disk.
- SPLINK 4.x (Python-native).

- Python 3.11, pandas, numpy, scikit-learn for metrics.
- Runtime: Local layer prediction 1-20s depending on config; global aggregation negligible.

5 Experimental Results & Discussion

5.1 Experiment 1: base Synthetic Dataset - `splink_local_demo.ipynb`

5.1.1 Dataset Description

This dataset consists of two tables with approximately 3,300 and 3,700 records, generated with 50 entities each, 30% overlap, and 70/80 records per entity (A/B). It serves as our primary artificial dataset for evaluating the Splink pipeline while maintaining computational tractability.

5.1.2 Blocking Rules Design

The blocking rules target orthogonal signals to bound recall while reducing the comparison space. Each rule is a disjunctive condition that allows matches when the attribute agrees or when data is missing on either side. The four blocking rules used are:

1. Categorical equality or missing
2. Numerical within 25% or missing
3. Date within +1 day or missing
4. Date within -1 day or missing

(See notebooks for the exact SQL predicates.)

5.1.3 Comparison Levels

Each field is modeled with a `CustomComparison` using binned levels that capture gradations of similarity. The levels are chosen to reflect the noise models used in synthetic data generation.

For the numerical field, we use `PercentageDifferenceLevel` bins at 5%, 10%, 25%, and 50%. For the categorical field, we use `ExactMatchLevel`. For the datetime field, we use `WithinNDaysLevel` bins at 0 days (same day), 1 day, and 5 days. For the textual name fields, we use `Jaro-Winkler` bins at 0.8, 0.5, 0.3. (See notebooks for full specification.)

5.1.4 Local Pairwise Matching Results

- **Match weight chart:** Figure 5 displays the m-probability, u-probability, bayes factor, and log2 bayes factor for each comparison level. This chart reveals which comparisons contribute most to the match weight.
- **Threshold selection curve:** Figure 6 shows precision, recall, and F1 score as functions of the match probability threshold. The curve illustrates the trade-off between precision and recall across different threshold values.

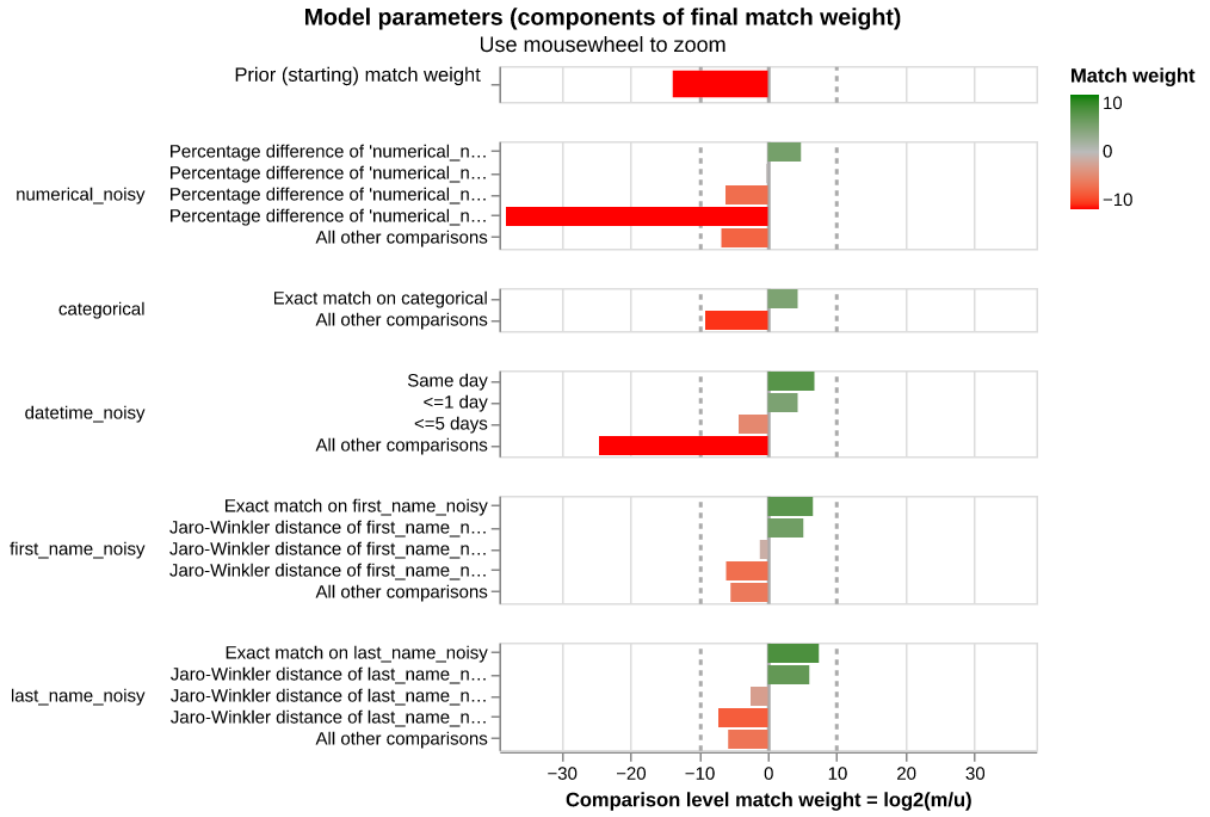


Figure 5: Match weights chart showing m-probability, u-probability, bayes factor, and log2 bayes factor for each comparison level in the small synthetic dataset.

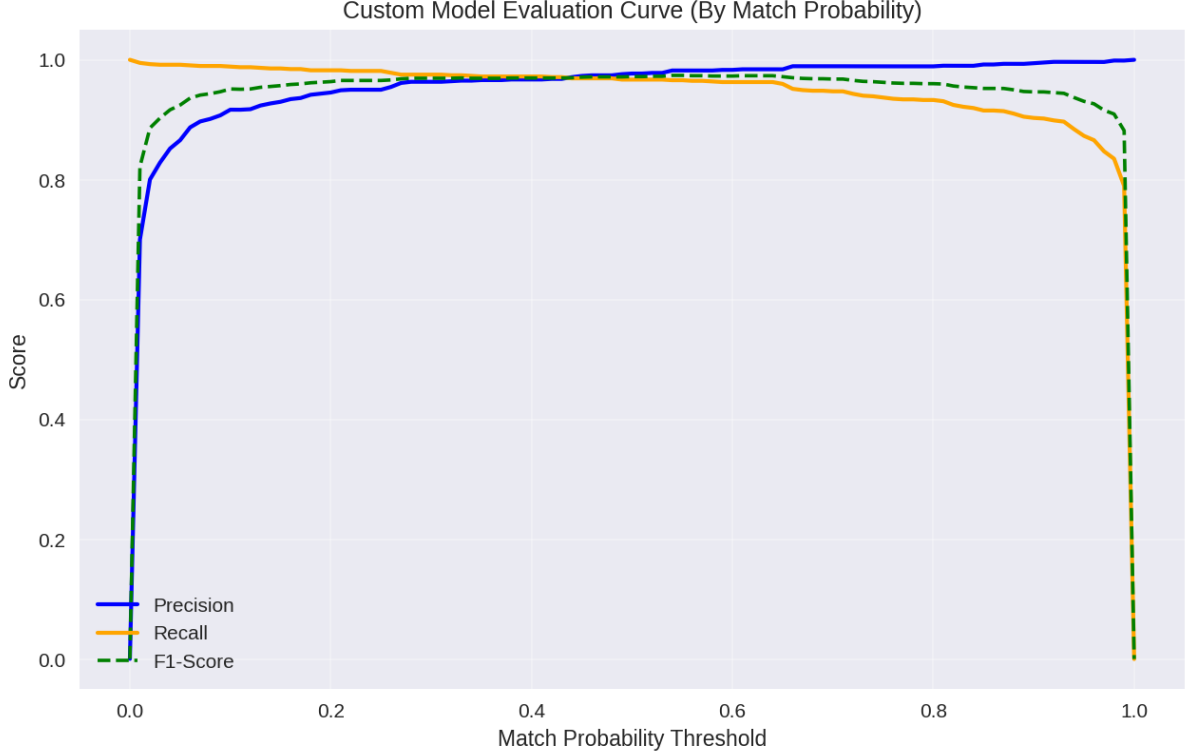


Figure 6: Threshold selection curve showing precision, recall, and F1 score as functions of the match probability threshold for the small synthetic dataset. The curve illustrates the trade-off between precision and recall across different threshold values.

5.1.5 Threshold Selection and Results

The intent behind threshold selection in these experiments is not to find an *optimal* threshold, but to ensure as large a **range of values** that would be considered *acceptable performances*. This is identical to spreading the gap between probabilities.

5.1.6 Local matching - Discussion

We notice from the match weights that the model seems to lean towards *heavy penalties* on mismatches over rewarding matches. Mismatches in numerical columns and the datetime are immediately penalised heavily.

Intuitively - these comparison-level mismatches are considered as effectively *equivalent* to a complete record mismatch.

5.1.7 Blocking Efficiency

With the same blocking rules, the number of comparisons after blocking is reduced from the full cartesian product of 3.48 million to 1,394,506.

5.1.8 Local-to-Global Aggregation Analysis

We apply local-to-global aggregation techniques to the base dataset by constructing entity pairs and computing various aggregation strategies that align with the theoretical framework taxonomy in Section 2. Specifically, we implement:

- **Match density** (`sum_p`) - see Section 2.3

- Max-Over-Partitions (max_w) - see Section 2.3
- Noisy-OR (score_noisy_or) - see Section 2.3
- Vector-Space-inspired (top_3_mean_w, score_softmax_p) - see Section 2.3
- Weighted Jaccard (weighted_jaccard) - see Section 2.3
- Weighted Overlap (weighted_overlap) - see Section 2.3
- Score-probability expected ratio (score_prob_expected_ratio) - see Section 2.3

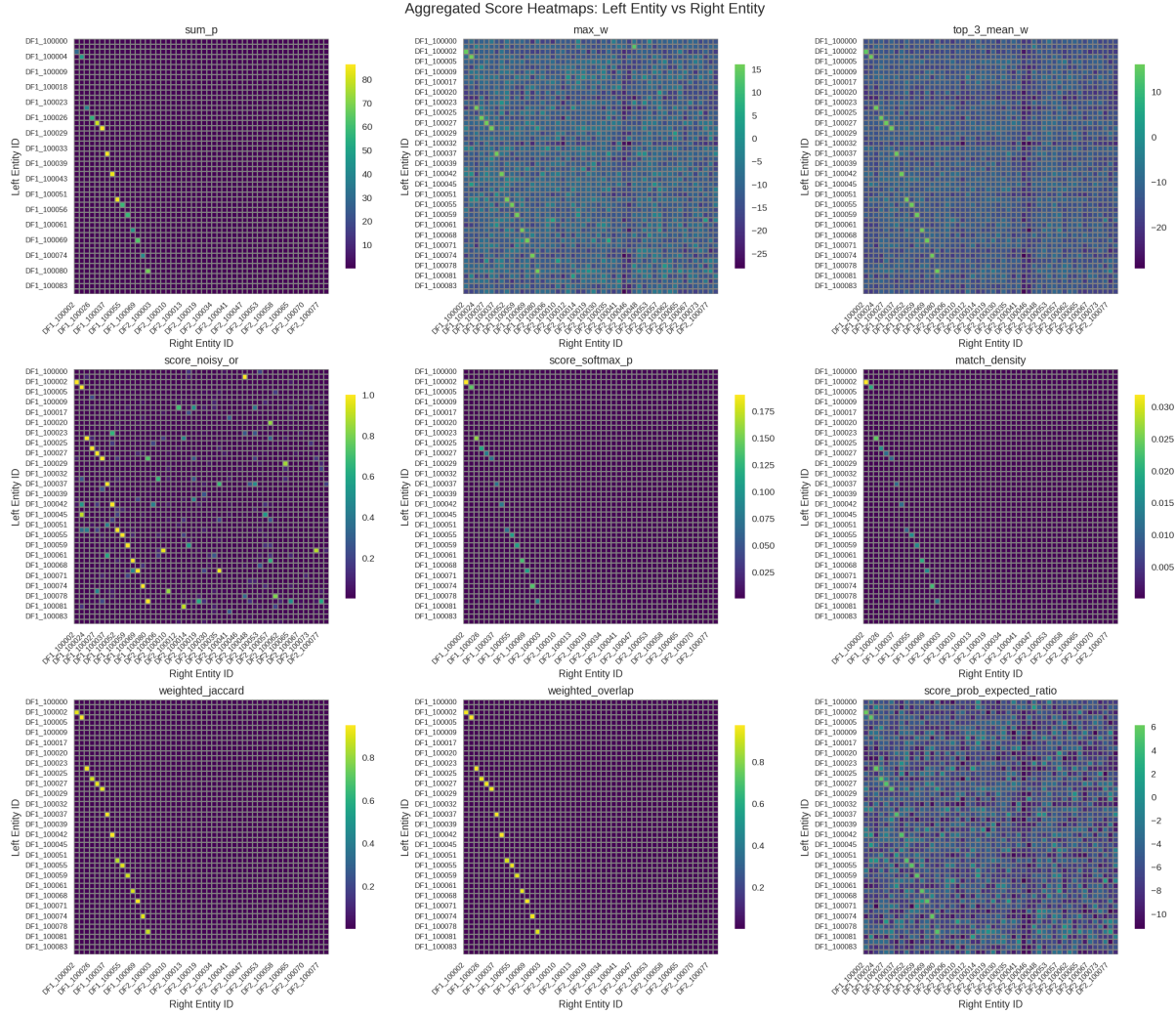


Figure 7: Aggregation score heatmaps showing various aggregation strategies (sum, max, top-k mean, noisy-or, softmax, match density, weighted Jaccard, weighted overlap, score-probability expected ratio) between entities in the base synthetic dataset, demonstrating the theoretical framework’s aggregation functions in practice.

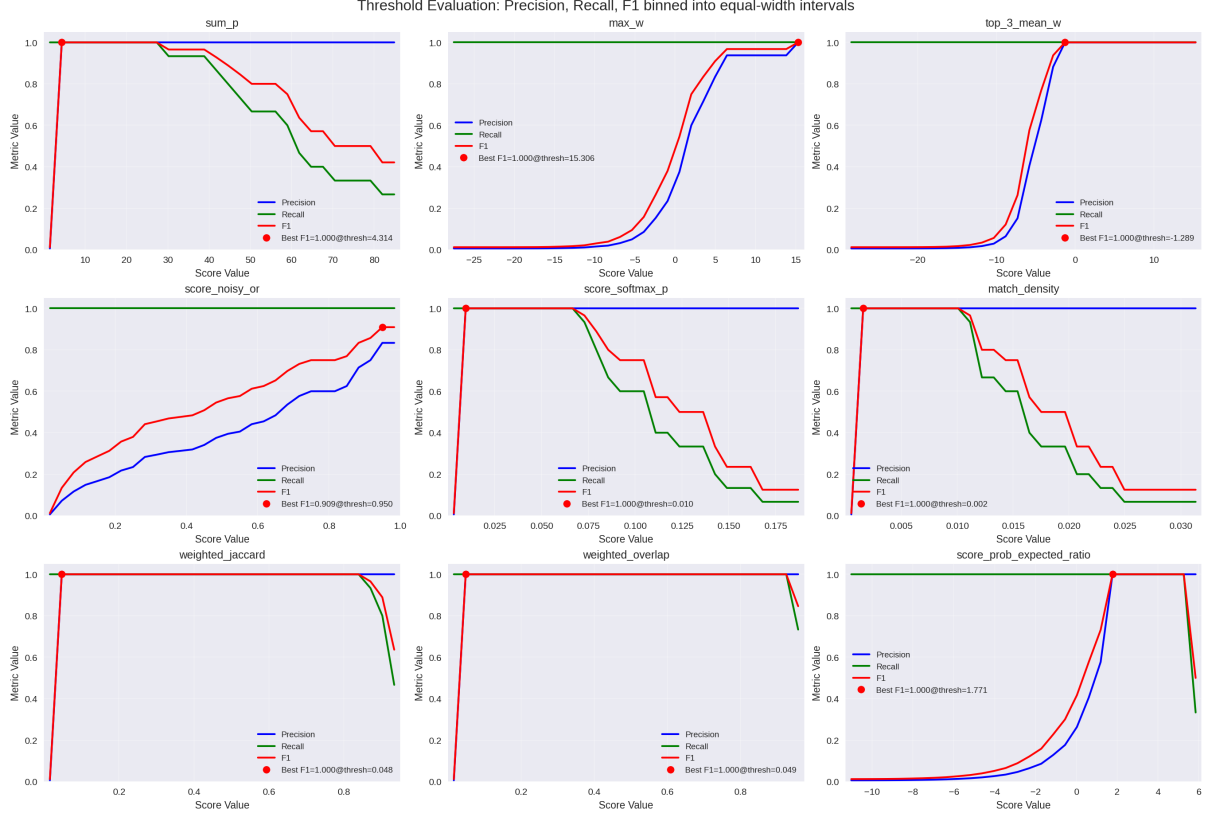


Figure 8: Threshold evaluation curves (precision, recall, F1 binned into equal-width intervals) for each aggregation strategy on the base synthetic dataset, demonstrating performance across the theoretical framework’s aggregation functions.

5.1.9 Global Aggregation - Discussion

We term the threshold curves for each aggregation function as *profiles*. We note that for a few of the profiles, namely the weighted Jaccard and the weighted overlap, the profile is generally stable - the recall and precision are not volatile. Furthermore, the scores on these functions are also exceptional - at recall 1.0 and precision 1.0 for a large range of these values.

5.2 Experiment 2: Synthetic Dataset Variants

To understand how specific dataset characteristics affect matching quality and computational efficiency, we created several variants of the synthetic dataset by modifying single parameters while keeping the Splink configuration constant. This allows us to isolate the impact of each factor.

5.2.1 Variant Overview

- **smallest**: This dataset consists of two tables with approximately 3,300 and 3,700 records, generated with 50 entities each, 10% overlap, and 35/45 records per entity.
- **low_card**: 100 entities each, 10% overlap, 18/12 records per entity (low intra-entity variation)
- **o50**: 50 entities each, 50% overlap, 30/35 records per entity

5.2.2 Configuration

All variants use the identical Splink configuration as Experiment 1 (same four blocking rules and three comparisons).

5.2.3 Results Summary

Each variant's results and metrics are to be extracted from their respective notebooks:

- `splink_local_smallest.ipynb` (smallest)
- `splink_low_card.ipynb` (low_card)
- `splink_o50.ipynb` (o50)

5.2.4 Intent

The variant experiments reveal how different dataset characteristics influence the EM algorithm's convergence and the final matching quality:

- **Overlap percentage:** Higher overlap (as in `o50`) introduces an error in the estimated prior - we show that selection via *ranking* still maintains performance.
- **Entity count:** Increasing entities (as in `smallest` against `base`) improves estimation stability but increases computational cost.
- **Cardinality:** Higher intra-entity variation (as in `base` against `low_card`) improves discrimination at every level.

5.2.5 smallest Variant

Dataset Description The smallest variant has 50 entities each, 10% overlap, and cardinality 35/45 per entity.

Results Figures 9 and 10 show the match weights and threshold selection curve for this variant.

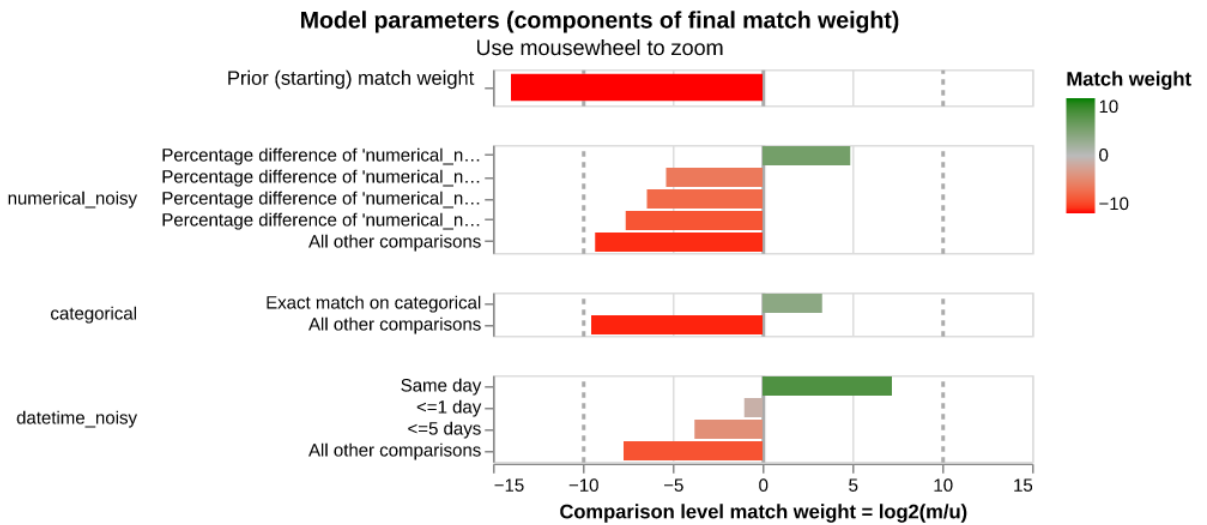


Figure 9: Match weights chart for the smallest variant (50 entities, 10% overlap, 35/45 cardinality).

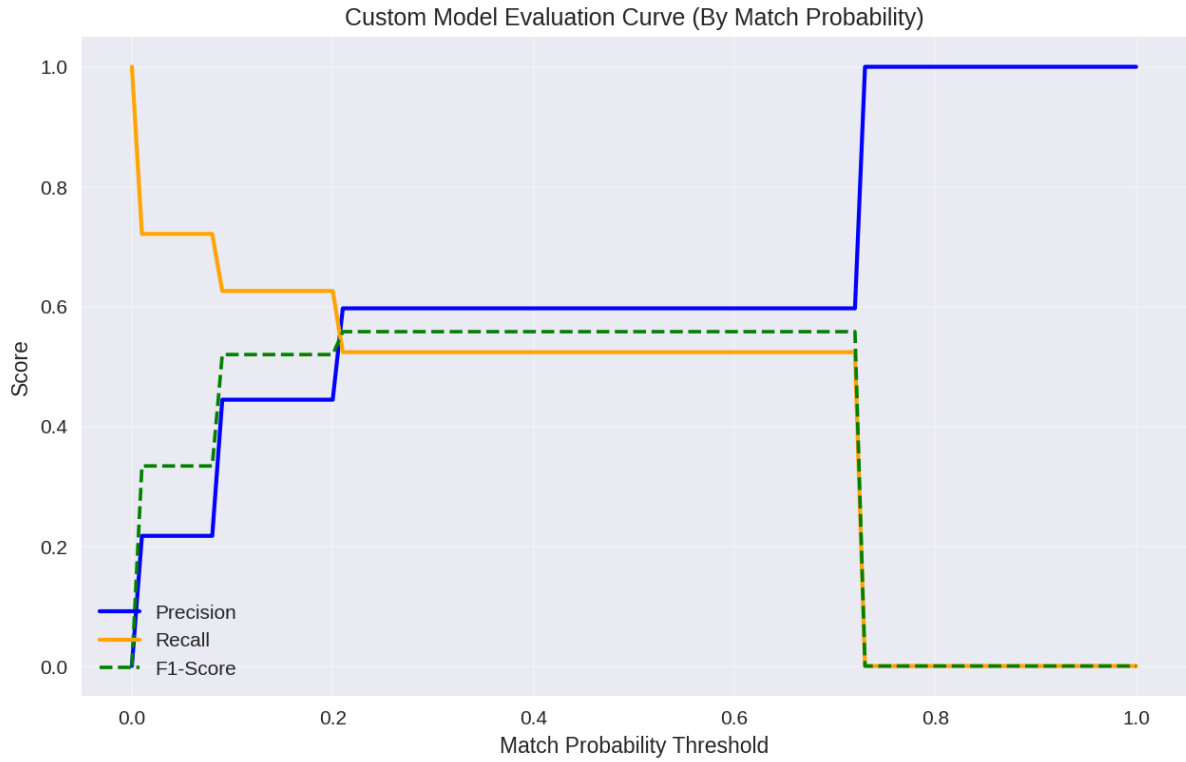


Figure 10: Threshold selection curve for the smallest variant.

5.2.6 Local-to-Global Aggregation Analysis

We apply identical local-to-global aggregation techniques to the smallest variant dataset.

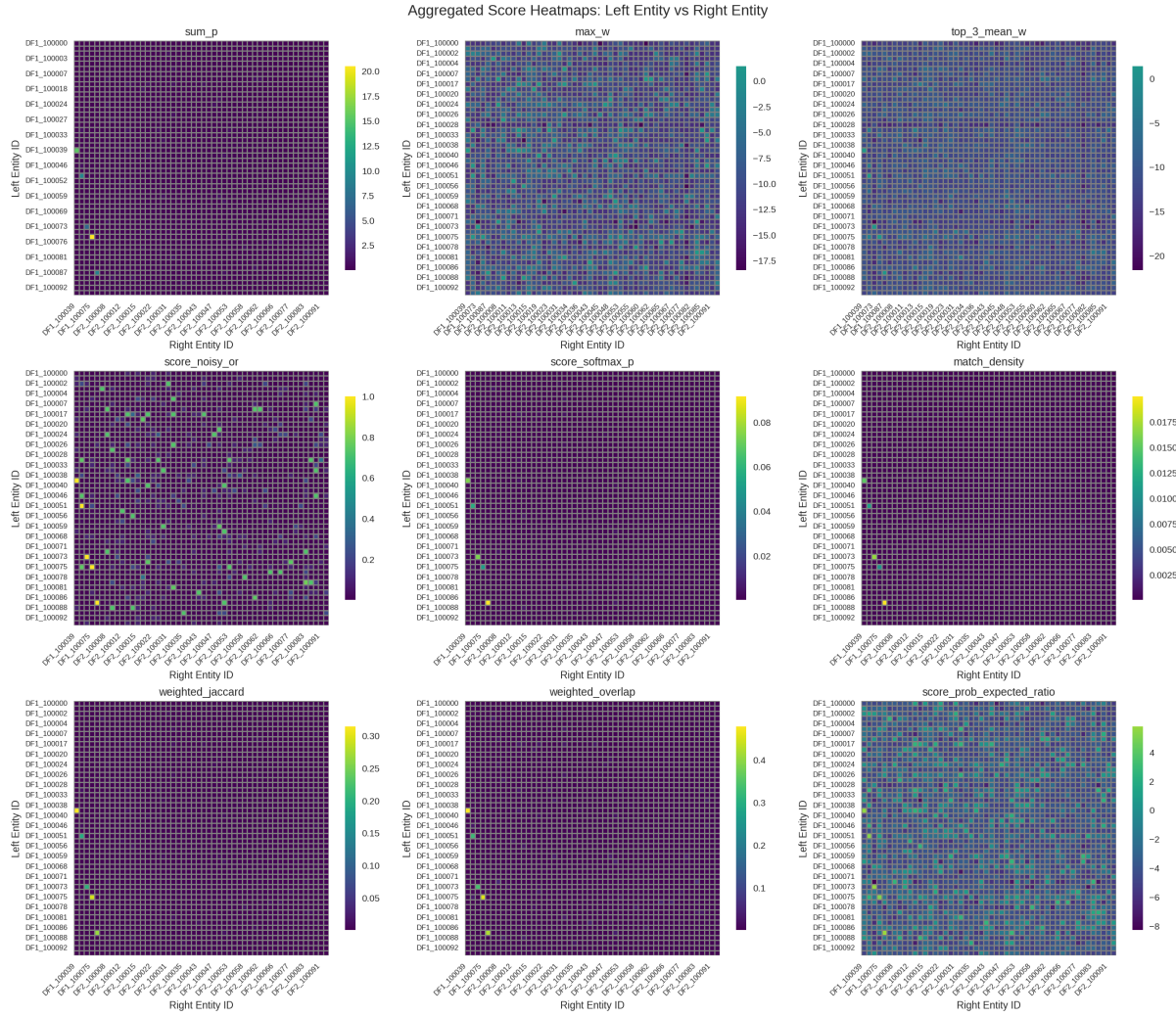


Figure 11: Aggregation score heatmaps showing various aggregation strategies (sum, max, top-k mean, noisy-or, softmax, match density, weighted Jaccard, weighted overlap, score-probability expected ratio) between entities in the smallest variant dataset, demonstrating the theoretical framework’s aggregation functions in practice.

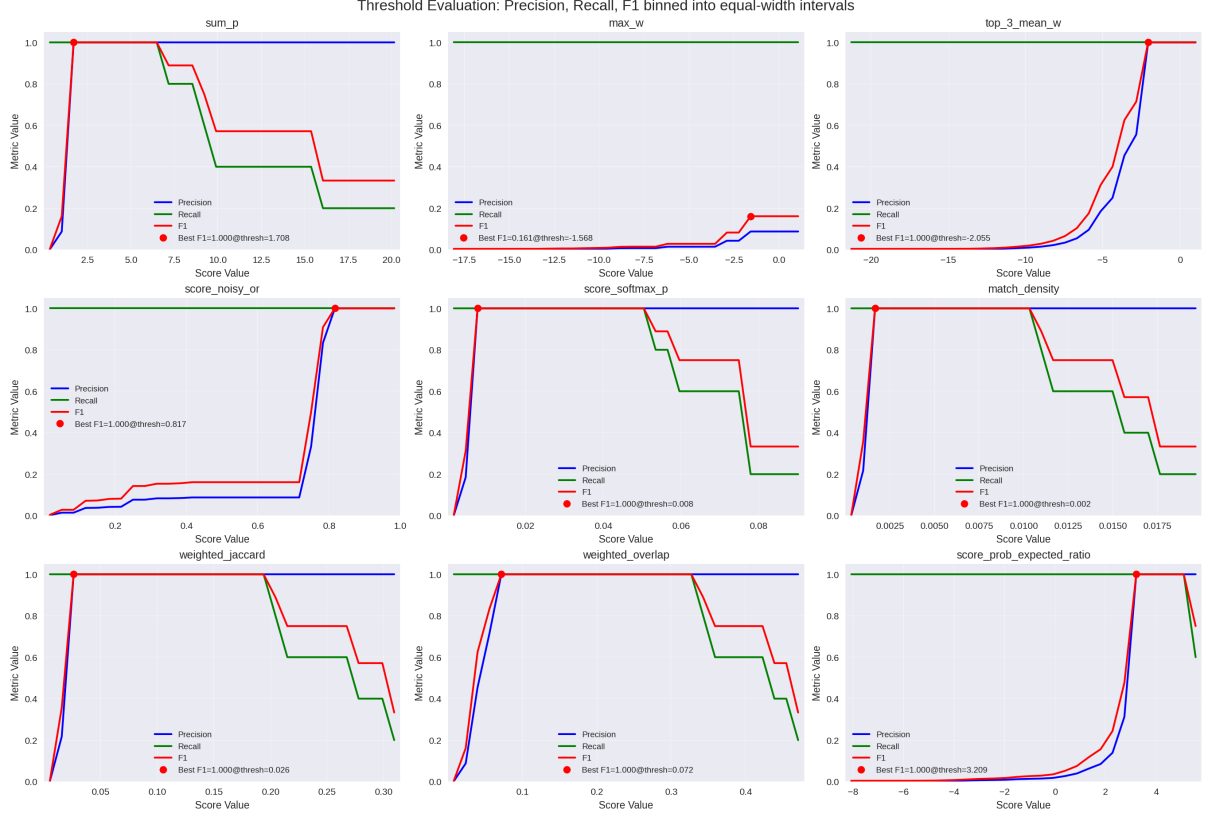


Figure 12: Threshold evaluation curves (precision, recall, F1 binned into equal-width intervals) for each aggregation strategy on the smallest variant dataset, demonstrating performance across the theoretical framework’s aggregation functions.

Discussion We notice that without the textual name columns, our model performs significantly worse on pairwise matching. Most of these comparison matches are still susceptible to coincidental matches, which result in lower match weights even on matches.

We penalise heavily, as expected, on mismatches as we have previously, but the lack of more discriminatory columns affected local matching significantly.

However, when extended to the global aggregate scores, our performance, while indeed poorer, still retains a satisfiable level of stability. In particular, weighted Jaccard and weighted overlaps retain similar profiles to that of the base variant.

5.2.7 low_card Variant

Dataset Description The low_card variant has 100 entities each, 10% overlap, and cardinality 18/12 per entity (low intra-entity variation).

Results Figures 26 and 27 show the match weights and threshold selection curve for this variant.

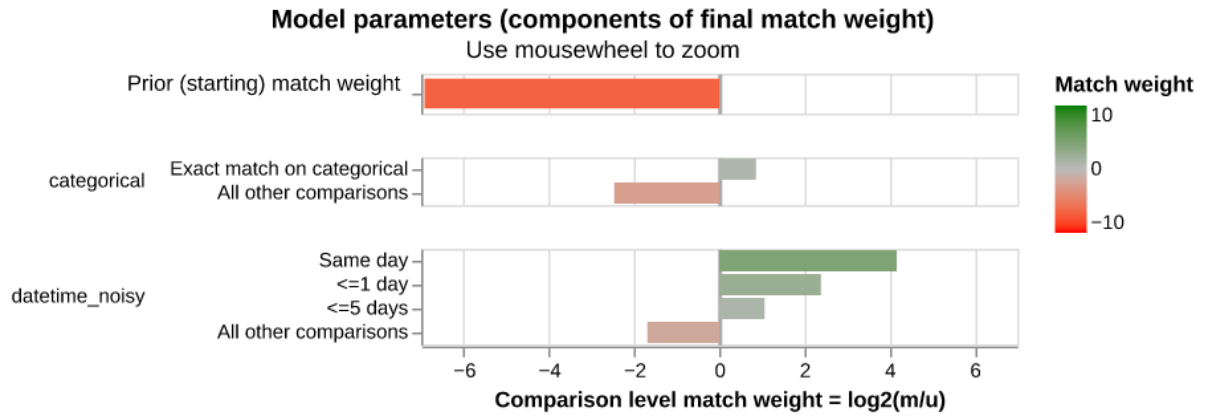


Figure 13: Match weights chart for the low_card variant (100 entities, 10% overlap, 18/12 cardinality).

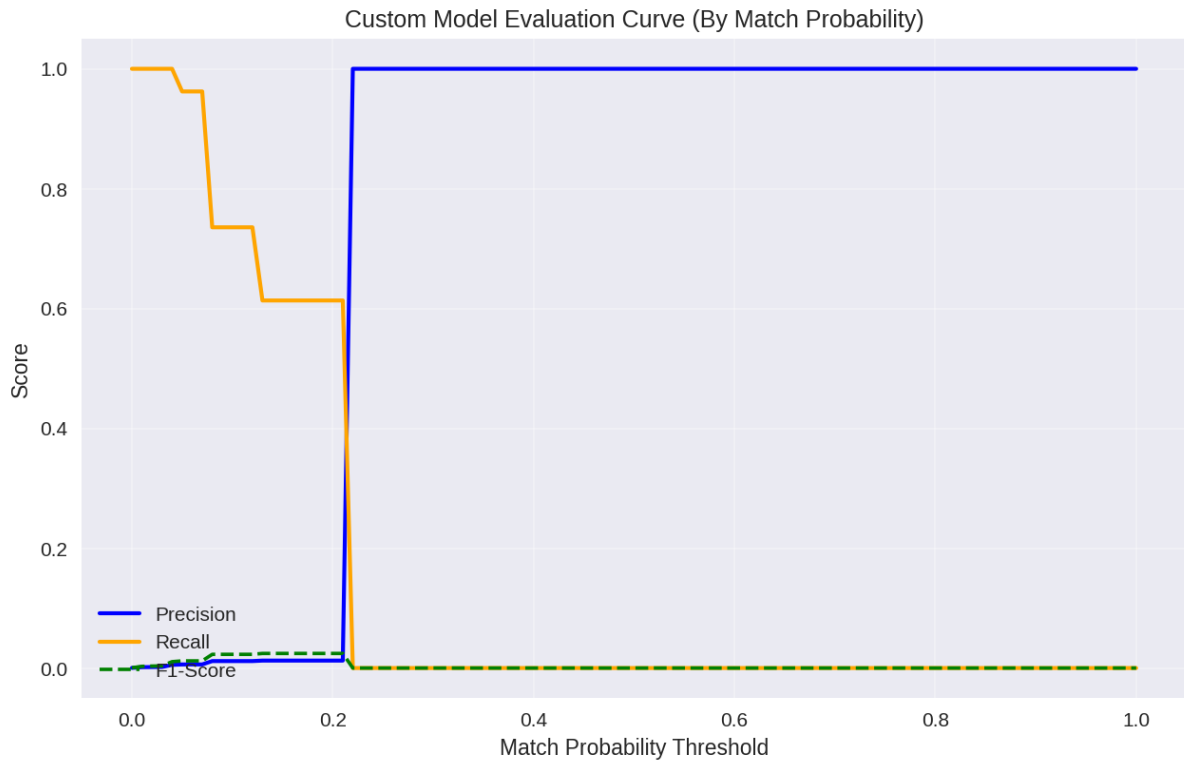


Figure 14: Threshold selection curve for the low_card variant.

5.2.8 Local-to-Global Aggregation Analysis

We apply identical local-to-global aggregation techniques to the low_card variant dataset.

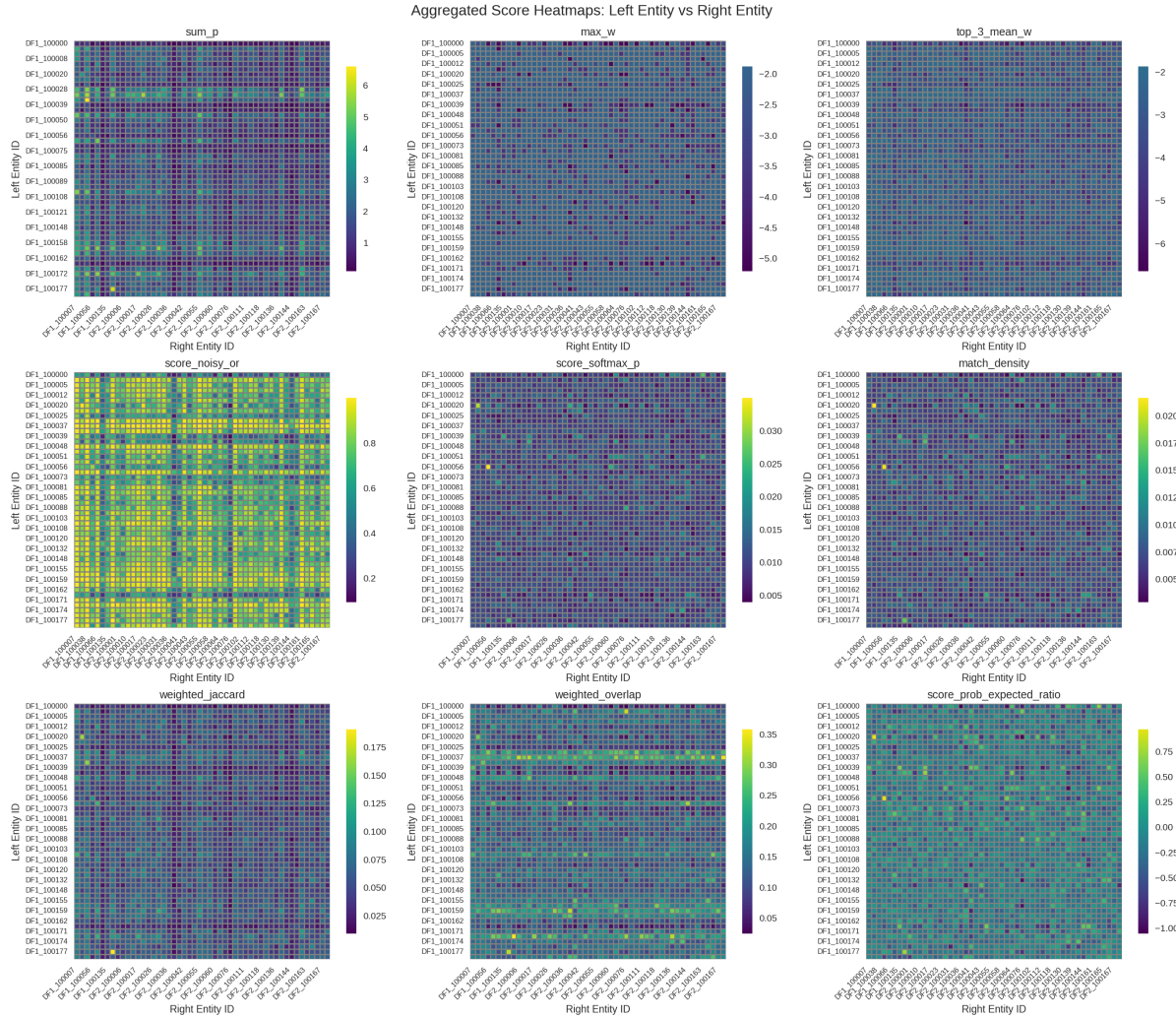


Figure 15: Aggregation score heatmaps showing various aggregation strategies (sum, max, top-k mean, noisy-or, softmax, match density, weighted Jaccard, weighted overlap, score-probability expected ratio) between entities in the low_card variant dataset, demonstrating the theoretical framework’s aggregation functions in practice.

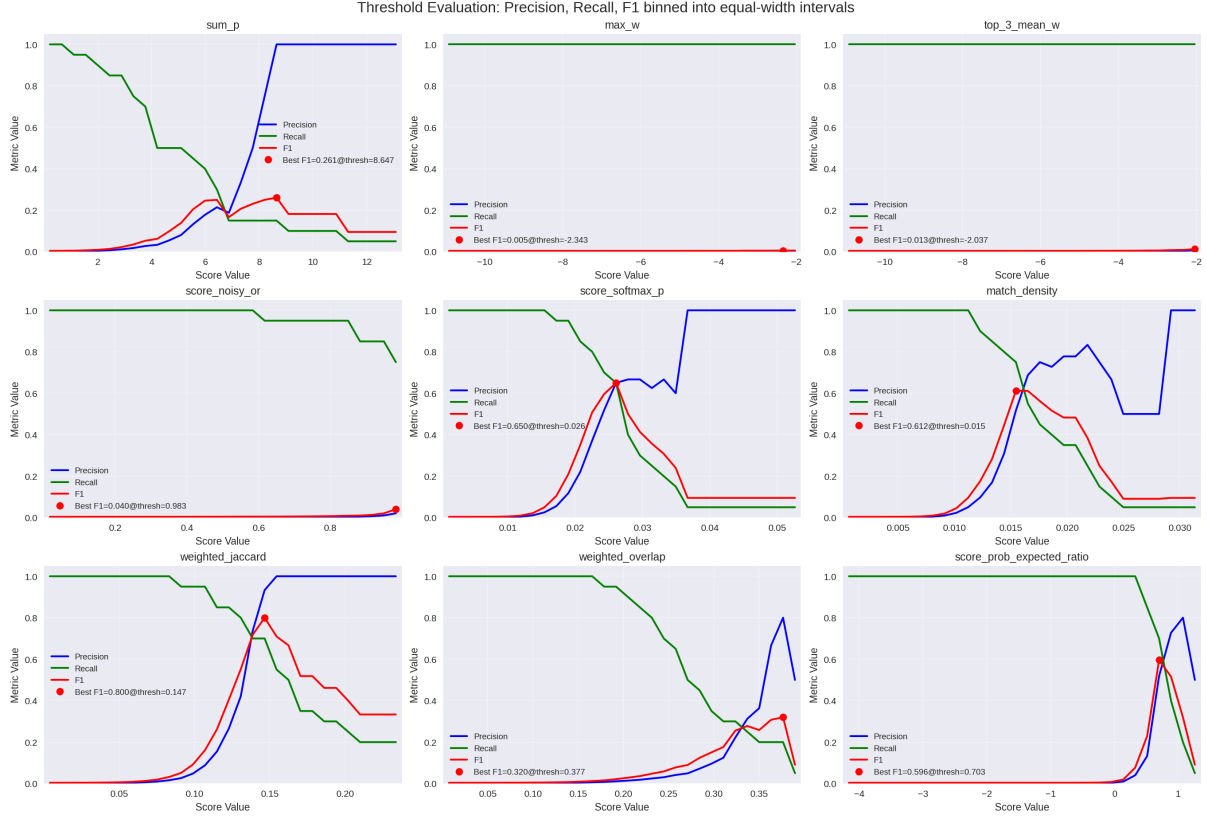


Figure 16: Threshold evaluation curves (precision, recall, F1 binned into equal-width intervals) for each aggregation strategy on the low_card variant dataset, demonstrating performance across the theoretical framework’s aggregation functions.

Discussion We find that the previous discussion from 5.2.5 finds its extremes here. Most comparisons are weakly discriminatory in nature, and we find that the model is unable to discriminate matches from noise.

Cardinality contributes to the discriminatory effect of Fellegi-Sunter. Here, we only have a total of 8 possible comparison vectors across the entire pair space, which creates little granularity for the necessary discriminatory effect.

This instability extends to the aggregation functions as well - the profiles on most, if not all, of the aggregation functions do not seem viable for stable use.

5.2.9 o50 Variant

Dataset Description The o50 variant has 50 entities each, 50% overlap, and cardinality 30/35 per entity (high prior match probability).

Results Figures 17 and 18 show the match weights and threshold selection curve for this variant.

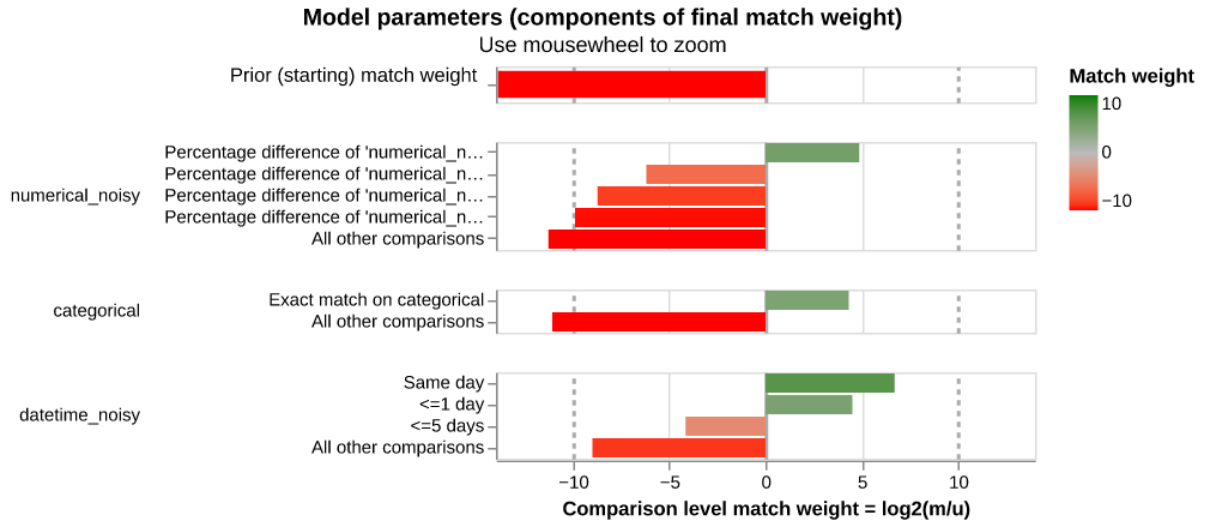


Figure 17: Match weights chart for the o50 variant (50 entities, 50% overlap, 30/35 cardinality).

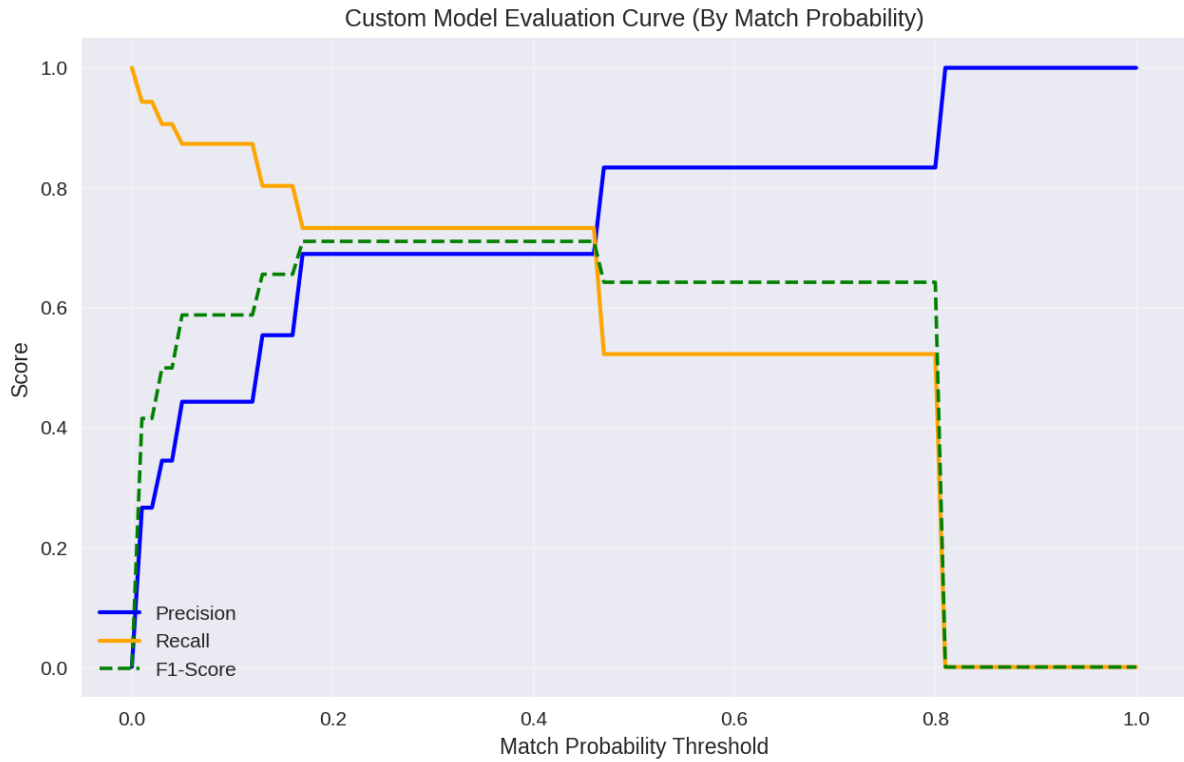


Figure 18: Threshold selection curve for the o50 variant.

5.2.10 Local-to-Global Aggregation Analysis

We apply identical local-to-global aggregation techniques to the o50 variant dataset.

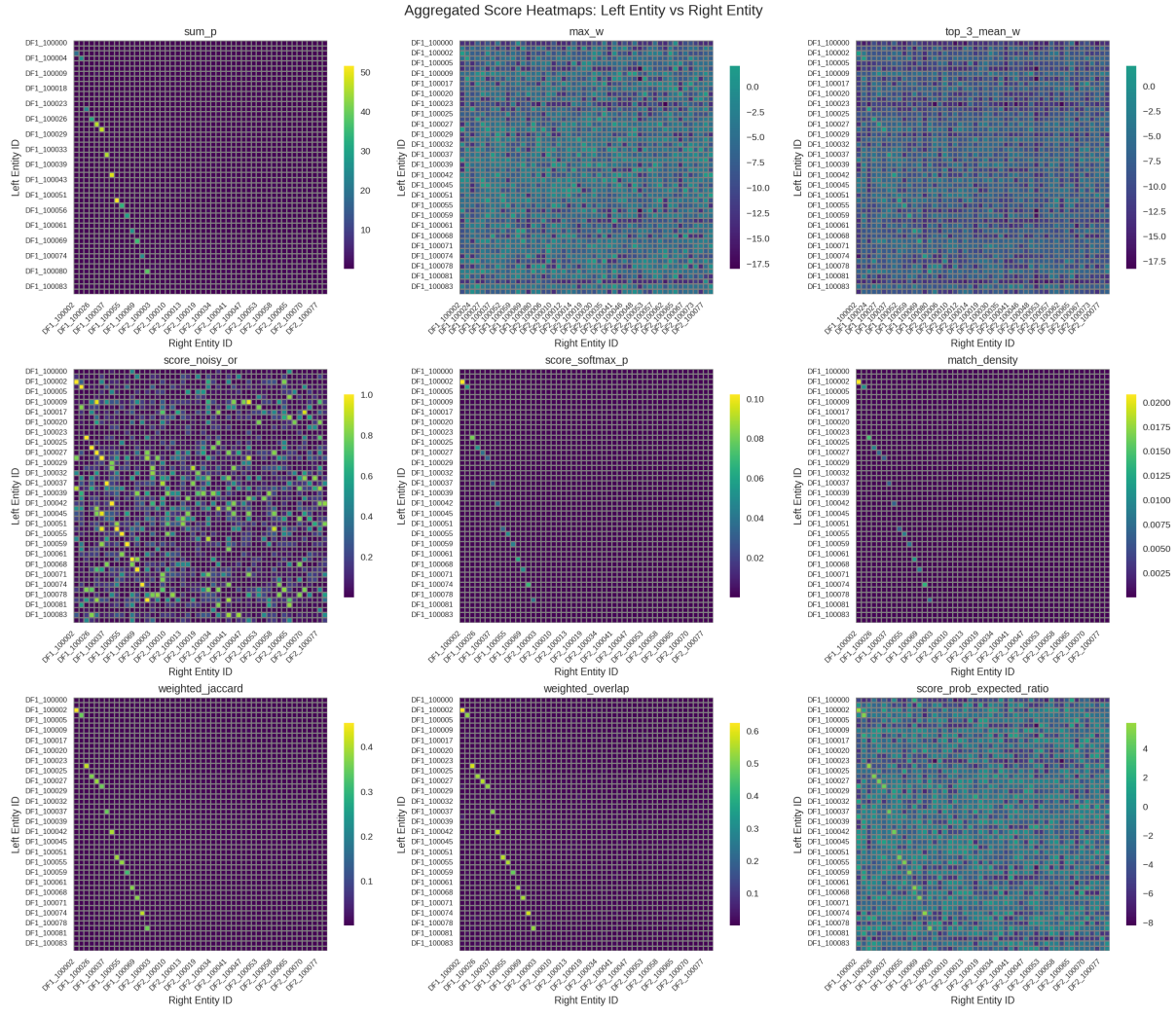


Figure 19: Aggregation score heatmaps showing various aggregation strategies (sum, max, top-k mean, noisy-or, softmax, match density, weighted Jaccard, weighted overlap, score-probability expected ratio) between entities in the o50 variant dataset, demonstrating the theoretical framework’s aggregation functions in practice.

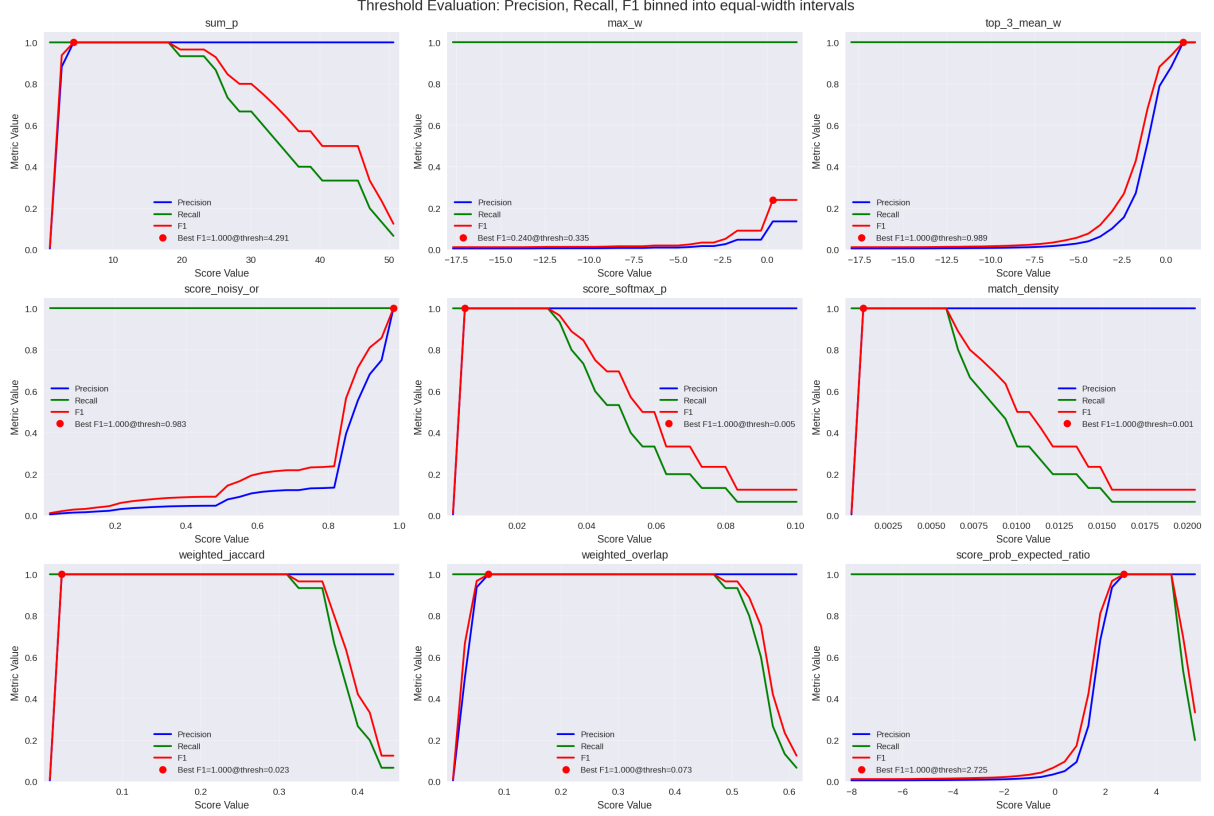


Figure 20: Threshold evaluation curves (precision, recall, F1 binned into equal-width intervals) for each aggregation strategy on the o50 variant dataset, demonstrating performance across the theoretical framework’s aggregation functions.

Discussion We find that the local matching performance of the model is somewhat similar to that of the smallest variant - albeit more granular. We attribute this step up due to having more ground truths.

We notice that the effect of wrongly estimating the recall when estimating the prior has little noticeable effects on model performance.

This is further shown when we extend to the aggregation functions - some of the threshold curves retain their profile, namely the weighted Jaccard and weighted overlap.

5.3 Experiment 3: Public Dataset Validation: MusicBrainz

5.3.1 Description

This experiment can be found in `splink_music.ipynb` and `splink_music_agg.ipynb`.

Two snapshots of the MusicBrainz dataset (catalog 3 vs catalog 5), each with approximately 3,900 and 3,800 artists (multi-record per entity). Attributes: title (set), album (set), track number (set), length (numeric), language (set), year (set).

5.3.2 Configuration

We block on at least one intersecting track number; comparison levels include soft Jaccard on title/album, Simpson on track number, year overlap, and numerical range overlap on length. The EM algorithm converges with 4 training rules.

Specifically, blocking rules are:

1. Same Artist AND Same Track Number
2. Same Track Number AND Same Title Prefix (Extended to 6 characters)
3. Exact Artist Match AND Title Prefix Match (first 4 characters)

Comparison levels include:

- Artist: Null, ExactMatch, JaroWinkler (threshold 0.88), Else
- Title Lexical: Null, ExactMatch, Prefix condition (`title_l LIKE title_r || '`
- Title Tokens: Null, Jaccard (0.80), Jaccard (0.55), Else
- Album: Null, ExactMatch, Album truncation match, Else
- Length: Null, Length within 15%, Length within 40%, Else
- Year: Standardized year list check using clean native DuckDB list operations (levels to be defined)
- Track Number: ExactMatch

5.3.3 Blocking Efficiency

The Cartesian product yields 15,028,728 comparisons. After applying the three blocking rules, the number of comparisons is reduced to 1,471 (a reduction of >99.99%).

Pre local-to-global matching Figures 28 and 29 show the match weights and threshold selection curve for the baseline Splink model (no aggregation).

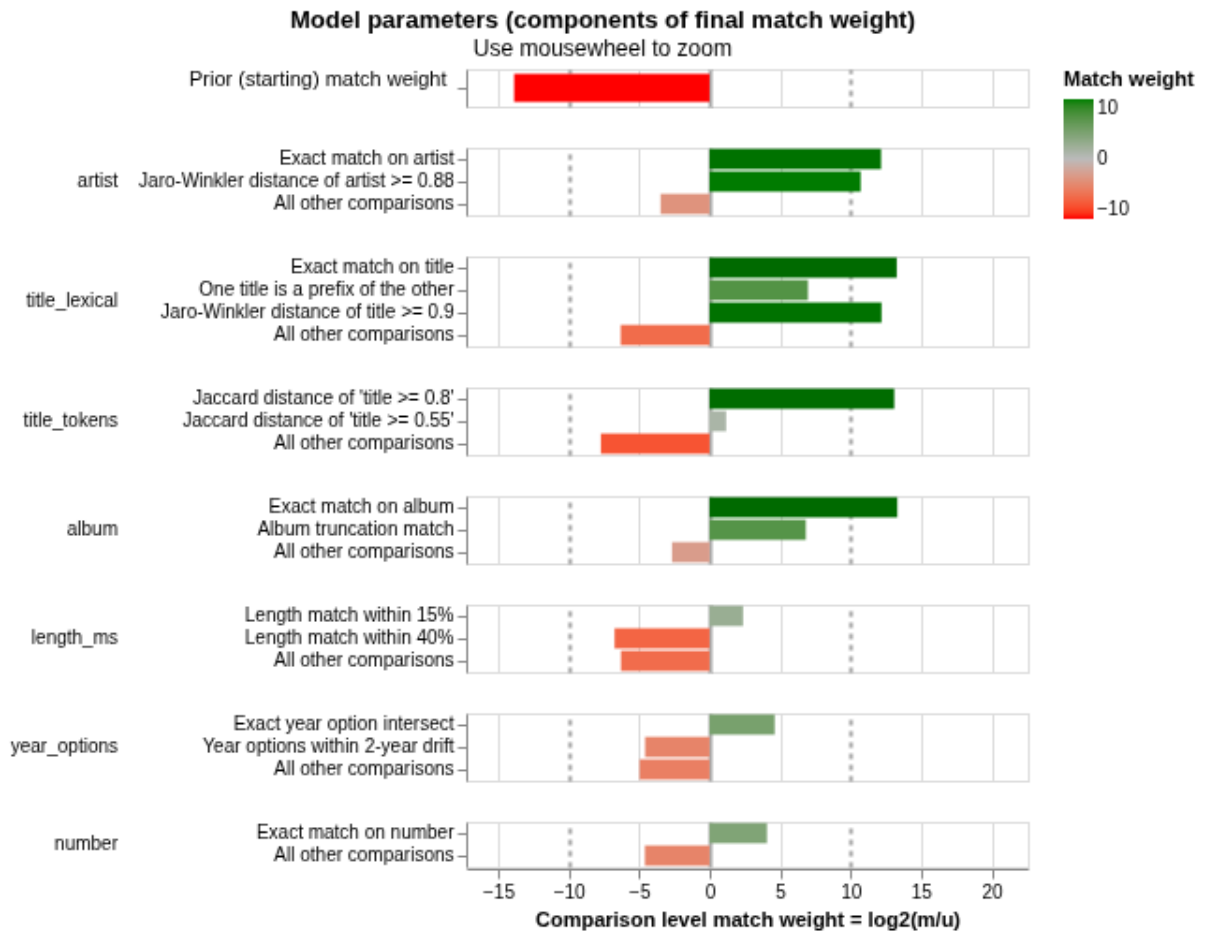


Figure 21: Match weights chart showing m-probability, u-probability, bayes factor, and $\log_2$ bayes factor for each comparison level in the MusicBrainz dataset.

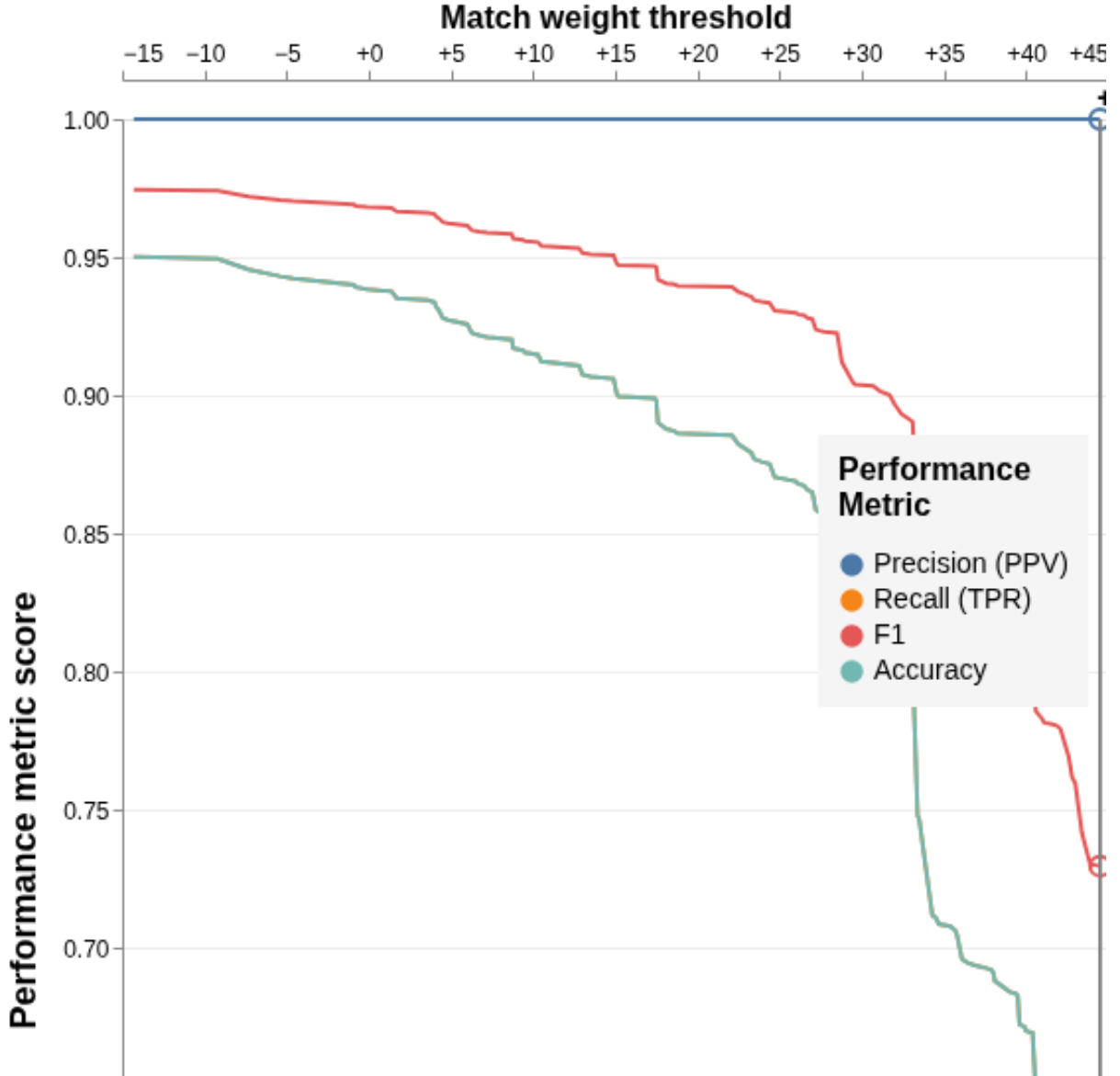


Figure 22: Threshold selection curve showing precision, recall, and F1 score as functions of the match probability threshold for the baseline Splink model on the MusicBrainz dataset.

5.3.4 Local-to-Global Aggregation Analysis

We apply local-to-global aggregation techniques to the MusicBrainz dataset by constructing entity pairs and computing various aggregation strategies that align with the theoretical framework taxonomy in Section 2. Specifically, we implement:

- **Match density** (`sum_p`) - see Section 2.3
- **Max-Over-Partitions** (`max_w`) - see Section 2.3
- **Noisy-OR** (`score_noisy_or`) - see Section 2.3
- **Score-probability expected ratio** (`score_prob_expected_ratio`) - see Section 2.3

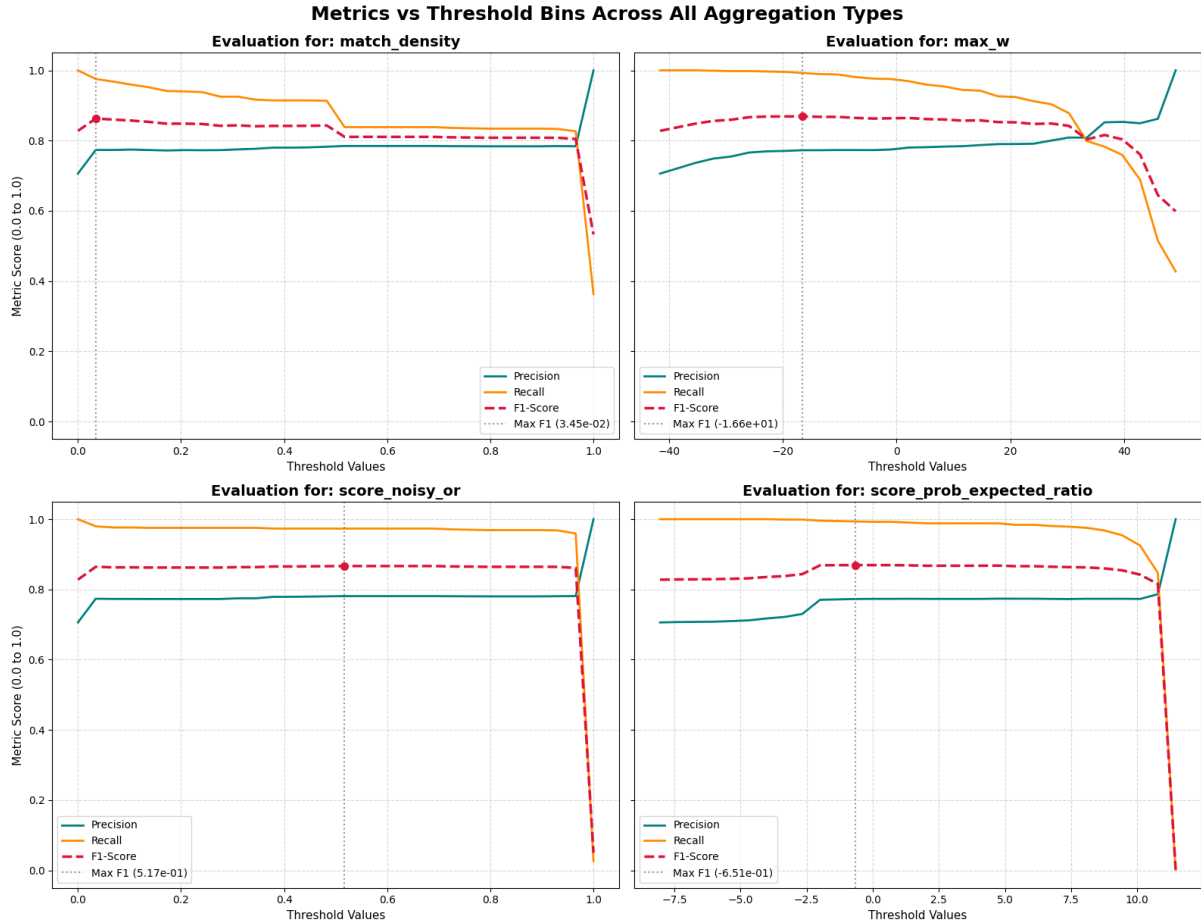


Figure 23: Threshold evaluation curves (precision, recall, F1 binned into equal-width intervals) for each aggregation strategy on the MusicBrainz dataset, demonstrating performance across the theoretical framework’s aggregation functions.

5.3.5 Discussion

The ground truths are established as a match on the artist column, acknowledging that some artists don’t actually have exact matches due to name typos (see Annex R for details on how this affects precision and recall). Under specific assumptions, we establish that precision is a definite lower bound, but recall is bounded to a fair estimate, and is likely an overestimation.

The MusicBrainz results confirm that the SPLINK pipeline, and the aggregation framework, generalizes to real-world, multi-record entity resolution scenarios. Local-to-global aggregation further enhances matching performance by leveraging distributional properties of match probabilities across record pairs.

We then discuss the stability of the global aggregation profiles, suggesting viability in using SPLINK for unsupervised matching in similar scenarios.

5.4 Experiment 4: Behavioural Aggregation Experiments

5.4.1 Description

This set of experiments builds profile vectors per global entity using numerical aggregates (mean, min, max of length) and categorical sets (title, album, language) converted to frequency vectors, then applies SPLINK to compare these profile vectors. We evaluate this approach on both our

synthetic datasets (base, and low_card) and the MusicBrainz dataset.

5.4.2 Configuration

The blocking rules and comparison levels are defined in the notebook they reference. General rule design concerning strictness of the blocking rules were applied.

The set comparisons are based on the attribute types (numerical, categorical, etc.) and are designed to capture similarity in the same way as the baseline experiments, but applied to the profile variables.

Base Dataset - agg_eg_normal.ipynb In this experiment, for simplicity, we do not use textual set-comparisons. We leverage on two key comparisons: magnitude-normalised cosine vectors, as well as the "bag" of tokens for each entity

Match weights We observe that the model relies extremely heavily on penalising poor matches - the likelihood of finding an exact match on these set similarity functions become much less likely.

We can build an intuition around this - the profiles of these entities are vastly different: the cosine vectors, as well as the "bag" of contents, are different.

Good matches can still be found by coincidence, but bad matches are immediately indicative, and equivalent, of a non-match overall.

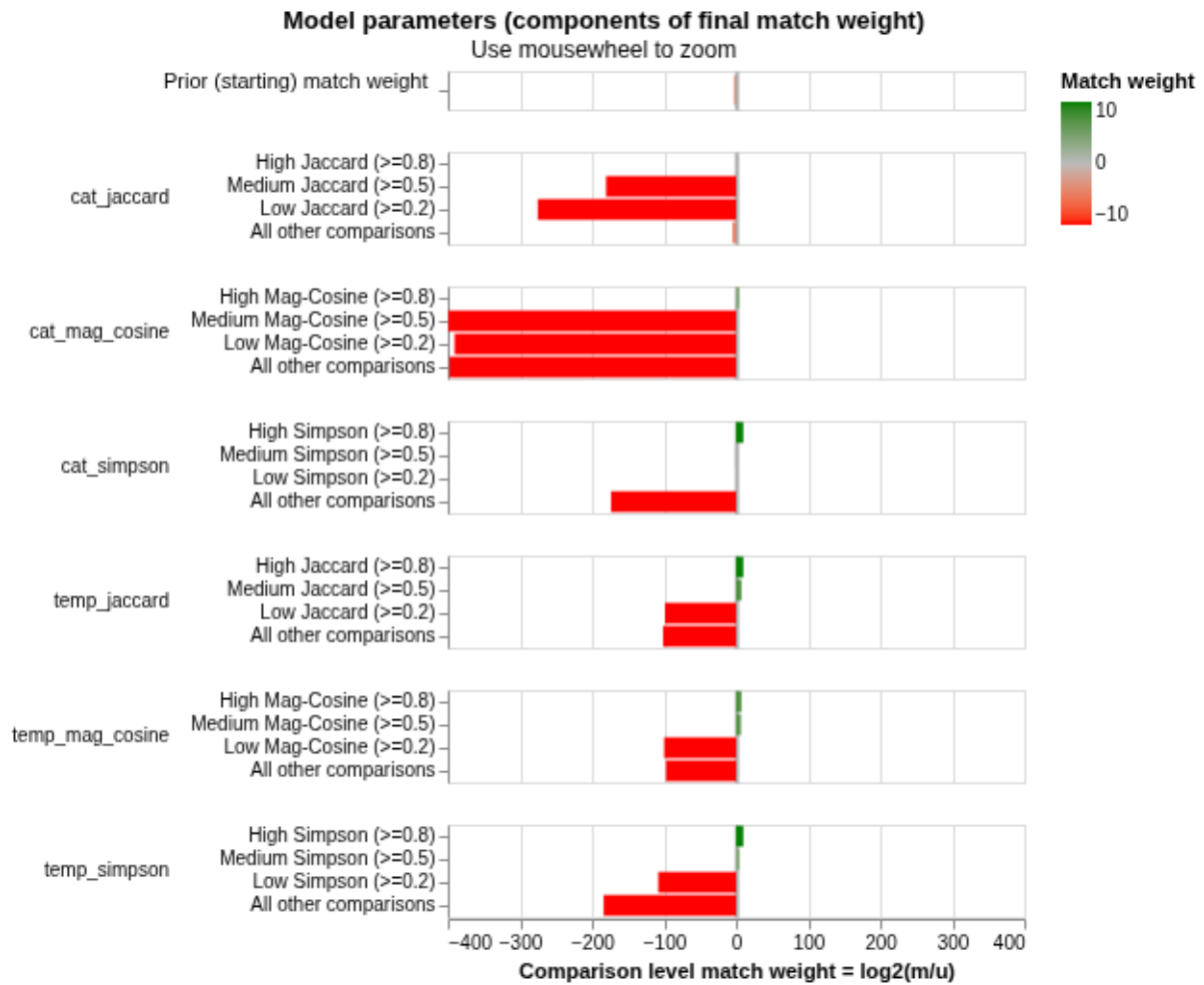


Figure 24: Match weights chart for the base variant dataset.

Threshold selection curve The profile here is extremely stable, and suggests promising results.

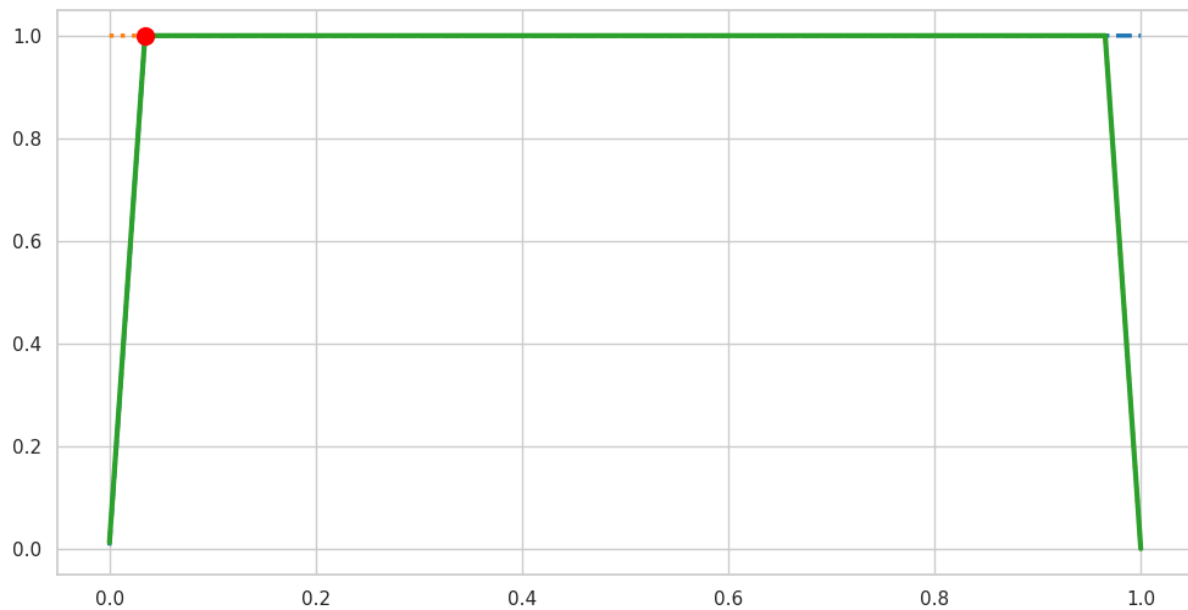


Figure 25: Threshold selection curve for the base variant dataset. The orange dotted curve is recall, blue dashed-line is precision. Green bold line is F1.

Low-card Dataset - `agg_eg.ipynb` We attempt find a better solution to global entity matching in profile aggregation with the `low_card` dataset.

Match weights The discriminatory factors on some of the set comparisons are strong - the model is able to pick up on mismatches between entities.

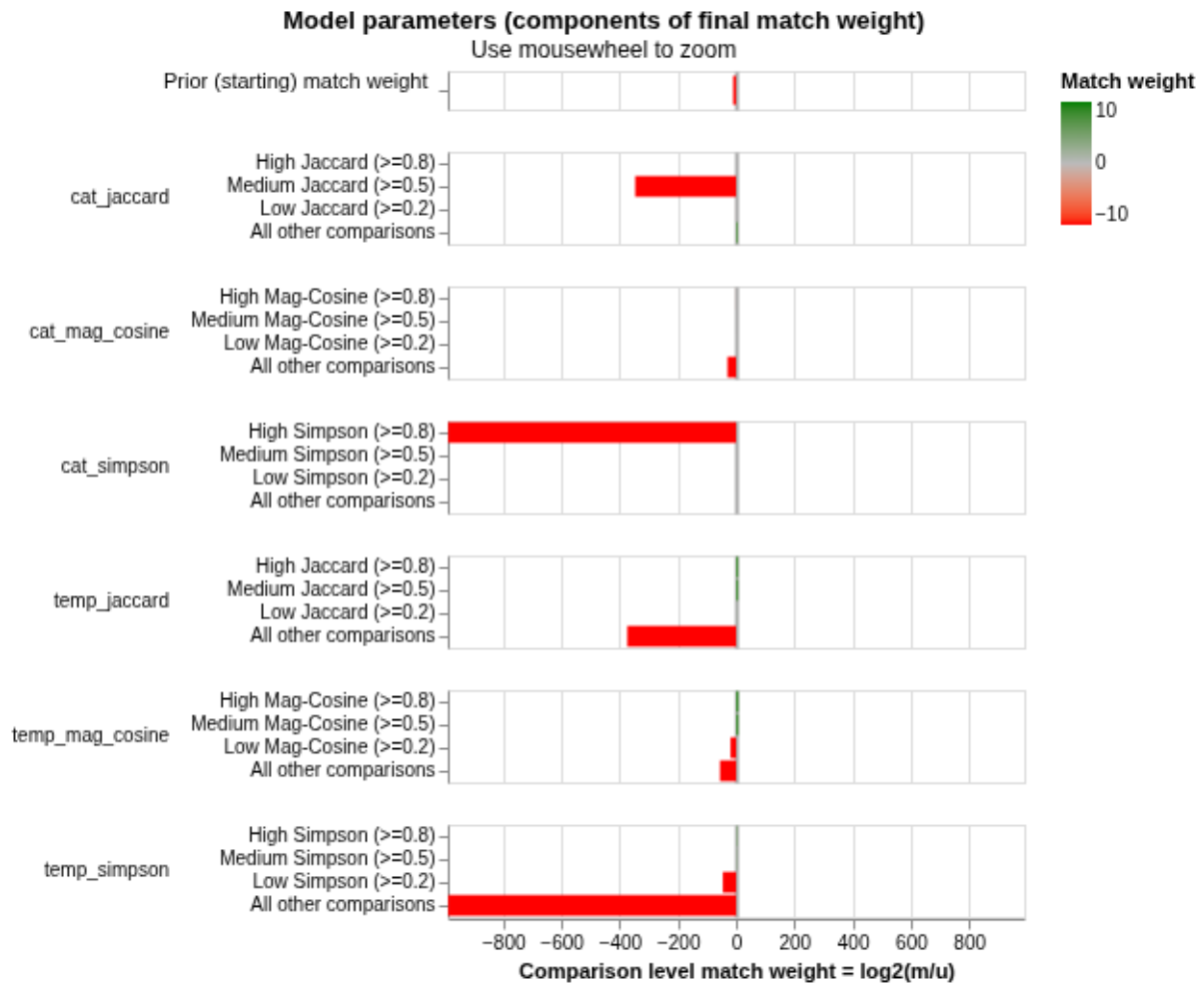


Figure 26: Match weights chart for the low_card variant dataset.

Threshold selection curve The profile is much more stable than observed from aggregation attempts.

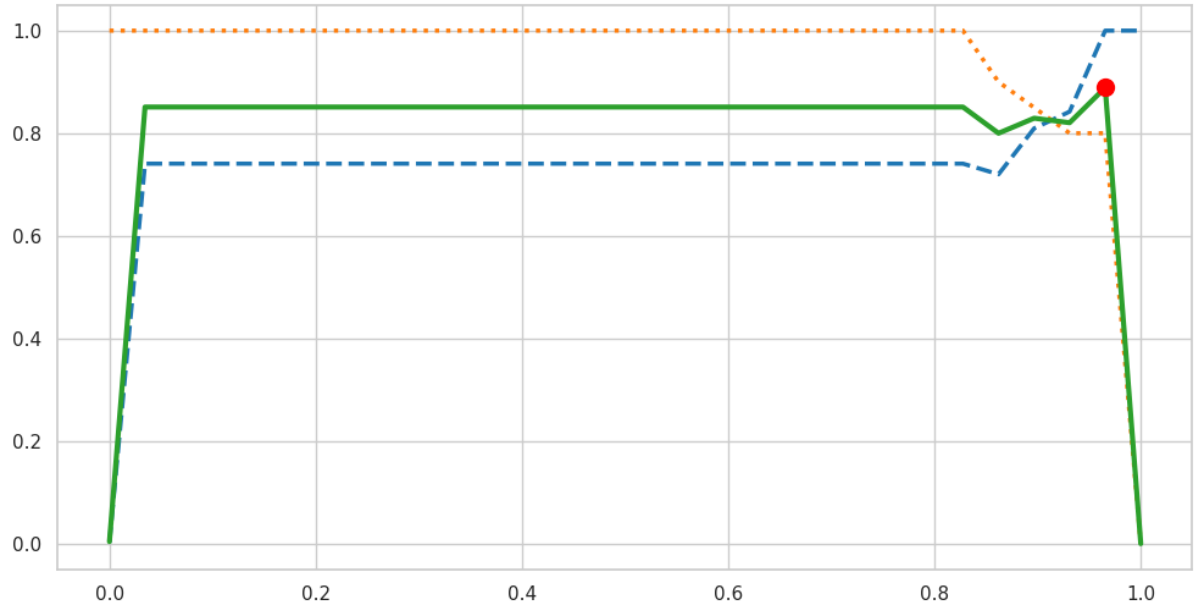


Figure 27: Threshold selection curve for the low_card variant dataset. The orange dotted curve is recall, blue dashed-line is precision. Green bold line is F1.

MusicBrainz Dataset - `agg_music.ipynb` We attempt to extend this approach to the music dataset, to explore its viability on real world datasets.

Match weights We find that the model penalises only soft Simpson mismatches (< 0.4) to a high degree.

Text based comparisons are difficult to configure in nature - however, the crux of the problem is that there are no indicative mismatches. We discuss this later.

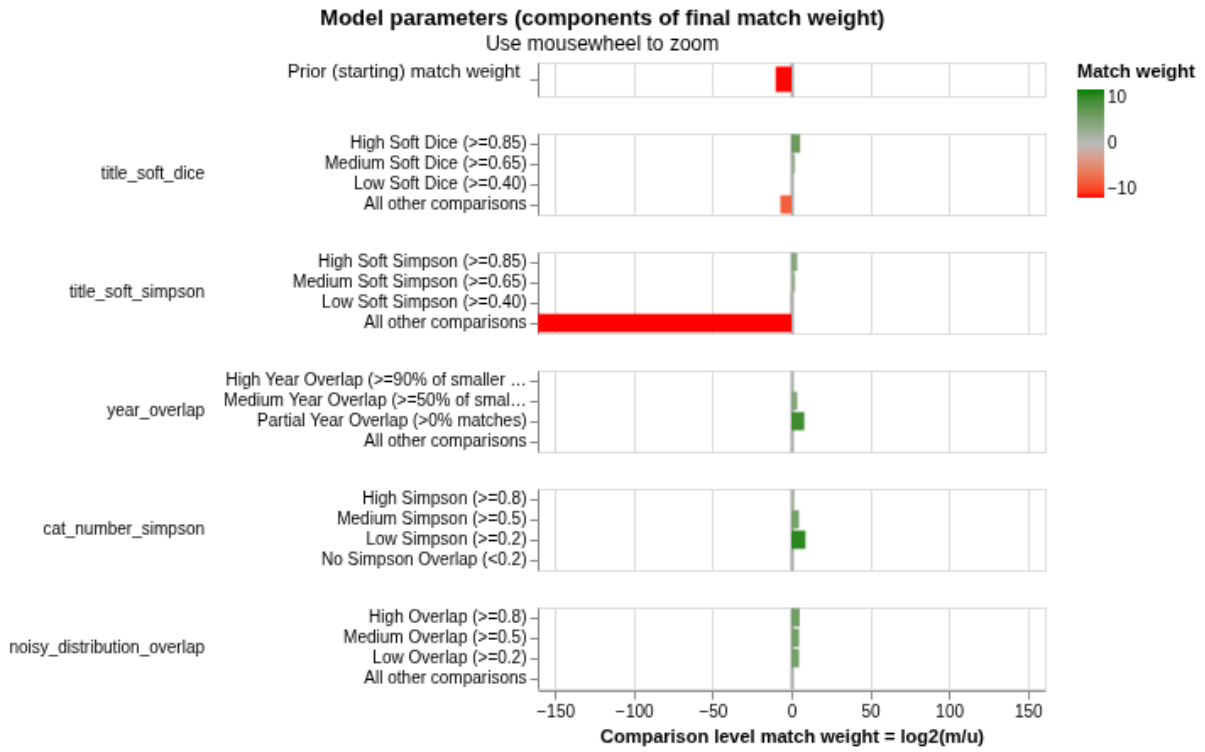


Figure 28: Match weights chart for the MusicBrainz dataset.

Threshold selection curve The lack of confident matches (or, indicative mismatches) propagates to instability in the threshold profile.

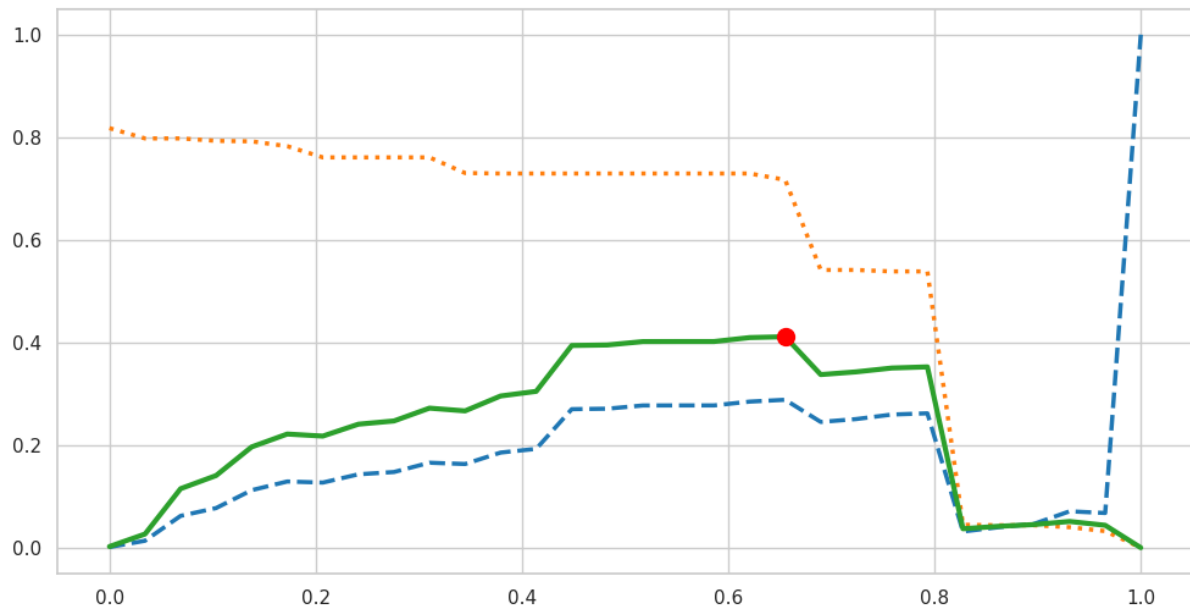


Figure 29: Threshold selection curve for the MusicBrainz dataset. The orange dotted curve is recall, blue dashed-line is precision. Green bold line is F1.

5.4.3 Discussion

The core approach in behavioural aggregation is in *profile matching*. In our synthetic dataset, the columns generated per row refer to a specific "event" - each of which has a unique, independent, distribution behind their generation.

In the MusicBrainz dataset, the individual records are *songs* - which skews the distribution of certain columns to a significant degree. The track numbers often WILL contain numbers from 1-5, and song duration distributions are often similar (2-4 minutes in length). These profiles are no longer unique - since every row is a song, every row's profile would be that *of* a song.

Most columns, in this case, provide effectively **no information** whatsoever to a profile match.

We find that the framework, can, and often does well, match profiles - but only in the specific case that entities are *uniquely profile-able*.

5.5 Experiment 5: Hierarchical 2-Layer Splink Approach

5.5.1 Description

This experiment explores a hierarchical approach where we first apply local SPLINK matching to generate match probabilities, then use these probabilities as features in a second-layer Fellegi-Sunter model. We demonstrate this approach using the `low_card` variant as an example.

5.5.2 Configuration

For the first layer, we use the identical Splink configuration as Experiment 1 (same four blocking rules and three comparisons). The match probabilities from the first layer are then used as input features for the second-layer Splink model, which employs a simpler configuration focused on learning weights for combining these probabilistic signals.

5.5.3 Results

From `splink_low_card.ipynb`, we extract:

- Hierarchical 2-layer Splink results: threshold curves showing precision, recall, and F1 scores across threshold values

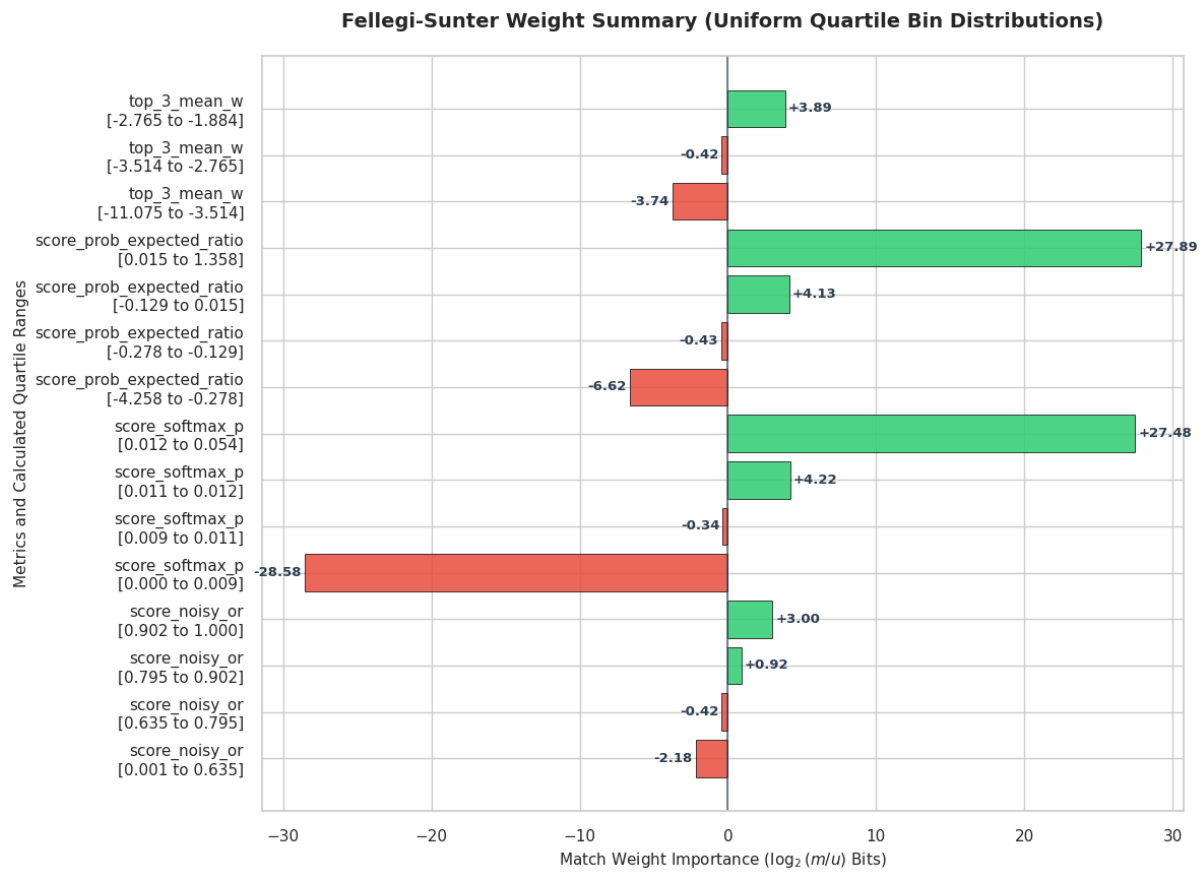


Figure 30: Match weights chart showing m-probability, u-probability, bayes factor, and log2 bayes factor for each comparison level in the hierarchical 2-layer Splink approach on the low_card variant.

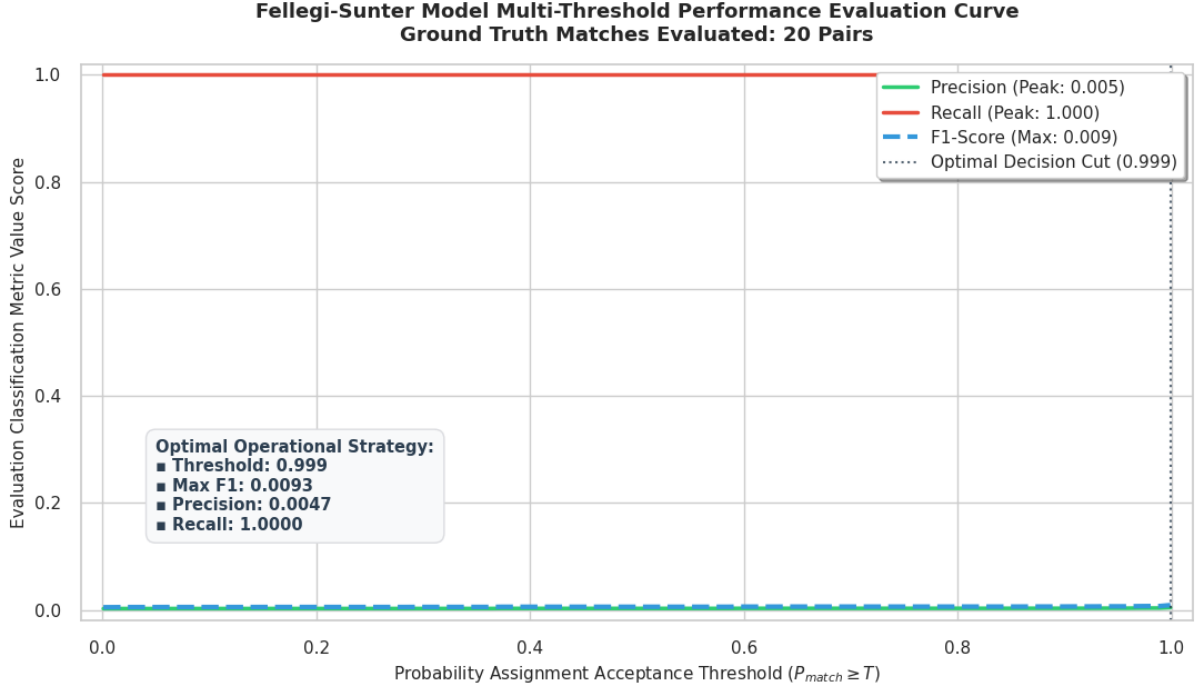


Figure 31: Threshold selection curve showing precision, recall, and F1 score as functions of the match probability threshold for the hierarchical 2-layer Splink approach on the low_card variant. The curve illustrates the trade-off between precision and recall across different threshold values.

5.5.4 Discussion

- **Polarising Effect:** As expected from the theoretical discussion in Section 2, the second layer exhibits polarising behaviour - the learned weights tend to extreme values, acting as “kingmakers” that dominantly select certain aggregation functions over others rather than learning balanced combinations.
- **Weight Interpretability vs. Practical Utility:** While the second-layer weights are explainable and indicate which aggregation functions the model considers most predictive, this extreme weighting renders the approach currently unviable for practical use (or, a viable dataset is yet to be found for this approach). The model effectively reduces to selecting a single dominant aggregation function rather than learning a nuanced combination.
- **Generalisation Challenge:** This behaviour highlights the need for further research into aggregation functions that are more generalisable and robust across different dataset characteristics. Fixed aggregation strategies may currently provide more stable performance than learned hierarchical weights.
- **Comparison to Direct Aggregation:** Initial exploration suggests that direct aggregation methods (sum, max, top-k mean, noisy-or, softmax) often provide more consistent and interpretable results than the hierarchical approach, which tends to overfit to the training signal.
- **Scalability Considerations:** While training two sequential Splink models increases computational complexity, the primary limitation is not computational but methodological - the hierarchical approach as currently formulated does not adequately generalise beyond the training distribution.

We further discuss, albeit at shallow depth, about a separate consideration - applying mathe-

matically simpler *confidence scores* in 7.

6 Edge Cases & Structural Extremities: Cardinality Breakdown

6.1 Failure Modes at Low Cardinality

The `low_card` config (18/12 rows/entity, 100 entities, 10% overlap) exposes fundamental failure modes:

6.1.1 Independence Assumption Violation

The FELLEGI-SUNTER model assumes conditional independence of comparisons given match status. At low cardinality, each GLOBAL ENTITY has few records ($m, n \approx 5$). The local pairwise comparisons between records of the *same* GLOBAL ENTITY are highly correlated—they share the same latent true values corrupted independently. Formally, for $r_{A,i}, r_{B,j} \sim e$:

$$\text{Cov}(\gamma(r_{A,i}, r_{B,j}), \gamma(r_{A,i'}, r_{B,j'})) > 0 \quad \text{for } i \neq i' \text{ or } j \neq j' \quad (23)$$

This inflates the effective evidence: the product of likelihood ratios overcounts the information. Result: match weights are overconfident (too high for matches, too low for non-matches), but the *ranking* may still be reasonable.

6.1.2 Combinatorial Explosion in Comparison Space

For a global pair with m and n records, there are $m \times n$ local pairs. When m, n are large (e.g., $18 \times 12 = 216$ pairs per global entity), the Cartesian product explodes. Blocking must be aggressive, but aggressive blocking loses recall.

6.1.3 Sparse Parameter Estimation

EM estimates $m_k(\gamma)$ from pairs in the blocking rule. At low cardinality, the number of true matches in any blocking rule is small ($O(N\rho)$ with small N). Parameter estimates have high variance; EM may converge to unstable values or fail to converge.

6.1.4 Aggregation robustness

As we see in 5.4, single scalar aggregation functions handle most cardinality regimes robustly. At regimes of extremely low cardinality, however, this approach breaks - record pairs are degenerate, and function scores are as a result, noisy.

7 Discussion

7.1 On confidence scores and the Fellegi-Sunter model

Remark: This subsection intends to address the following: Modern implementations of Fellegi-Sunter’s framework are *sufficiently discriminatory*. Confidence scores serve to apply *specific additional constraints* into our matching, which is better addressed through specific matching algorithms or approaches.

We can intuitively understand that given some sort of similarity/comparison function (for example, a percentage difference function between two values,) gives indication towards whether or not two records are similar. We also understand that it could be likely that two records might share quite a few different columns that are generally likely to be similar as well (it is quite likely that two identity records would match on gender, first name, etc.).

Requiring a discriminatory function: Thus, we wish to find a secondary function that differentiates scores by taking in *relative ranking* into account in regards to the best possible match we can find to a record. We define these as “confidence scores”, but in essence we simply require the following traits for this score:

1. **Normalised:** In order for this score to be comparable to other score values, this function must be normalised.
2. **Tie-breaking:** on the account that a record is so degenerate that it finds high similarities over various columns above some threshold on multiple records from the other dataset, we need to have some function that determines a tie-breaker on which match is stronger. Two general cases arise:
 - a) Record X from dataset 1 against A and B from dataset 2 forms the comparison vectors $[1, 1, 1, 0]$ and $[1, 1, 1, 0]$. However, this comparison vector is, in one way or another, the “best” in A, and not so in B. The score should be able to differentiate this. We also know this as Global Uniqueness.
 - b) Record X against A and B forms the comparison vectors $[1, 0, 0, 1]$ and $[1, 1, 0, 0]$. The confidence score must be able to determine which one of these are “better” matches. We also know this as feature weighting.

Initial consideration: confidence functions We consider the effects of applying some sort of confidence score. In this example, we discuss to the extent of using a softmax function, assuming we have a set of similarity scores across the attributes. We define the softmax function with non-directionality:

$$\text{Conf}_{\text{SM}}(e_i, f_j) = \frac{\exp(\text{sim}(e_i, f_j))}{\sum_{(e_i, f) \in A} \exp(\text{sim}(e_i, f))}$$

We first address the distinction in various confidence functions: z-score, relative ranking, etc. The relative order remains the same (by definition), only the *size* of the gap changes (Proof in the annex):

- Relative ranking enforces a “*equal penalisation*” constraint for varying scores.
- Z-score enforces a *linear* transformation of the differences.
- Softmax enforces an *exponential* transformation of the same differences.

We can imagine we would use these scores in two ways:

1. On individual attribute comparisons: softmax when uniqueness is of concern, z-score when attempting to filter noise. Here, we are effectively attempting to apply discriminatory match weights, targeting case b.
2. On a scalar aggregation function across the entire comparison vector. Here the distinction is less clear, but we are still attempting to discriminate and rank the same vectors in different distributions, targeting case a.

The enormously expensive computational cost aside, these implementations require **extensive manual intervention** and are **not generalisable** to a different set of records with different data structures.

A more elegant solution - Fellegi-Sunter: The core purpose of probabilistic matching is to discriminate attributes by their power to find a match. Feature weightage (case b) finds

its solution in m-probabilities (data quality / reliability) - certain attributes must have more reliability in discerning matches - the resultant match weight skews higher for these.

We now attempt to resolve case a: applying a confidence score destroys the globally-absolute probability already assigned by *enforcing additional constraints haphazardly*. (For example, a score of 0.4 in a pool of 0.1 would push itself to 1 - despite actually being a clear poor match).

- For local matching - this is elegantly solved by applying matching algorithms (Hungarian matching algorithm, Optimal transport). These algorithms enforce required constraints without adjustment to the probabilities. - For global, grouped entity matching - the crux of the report is the proposal of an approach that leverages aggregation: local score aggregation smooths noise, while profile based aggregation matches unique and consistent *behaviours*.

7.2 Splink, and modern Fellegi-Sunter

Splink is, in essence, the modern implementation of the Fellegi-Sunter probabilistic model that sets the benchmark standard for record linkage.

Beyond the application of the Fellegi-Sunter model, it also supports other well-researched heuristics and approaches, almost all of which became indispensable in this project’s empirical research:

- Term-Frequency based dynamic match weightage
- Blocking
- Fuzzy matches

The documentation is available at <https://moj-analytical-services.github.io/splink/index.html> [Linacre et al., 2024]. Sample usage is also provided in the repository linked.

7.3 Limitations

- **Synthetic data only partly represents reality:** The noise models (Gaussian, uniform categorical) are simplistic. Real data has systematic biases, format variations, schema drift.
- **No temporal evolution:** Entities are static; real ER often involves temporal linking (same entity at different times).
- **Two datasets only:** Multi-source (> 2) global resolution introduces transitive consistency constraints not addressed here.
- **Unsupervised only:** No exploration of semi-supervised (few labels) or active learning regimes.
- **Compute-limited:** DuckDB single-node; distributed SPLINK (Spark) not tested.

8 Future Work & Conclusion

8.1 Future Work

8.1.1 1. Automated Threshold Selection via Mixture Models

Currently, the decision threshold is selected by maximising F_1 on a clerical sample (synthetic) or via heuristic (public). A mixture model on the match weight distribution can automate this:

$$f(w) = \pi_M \mathcal{N}(w; \mu_M, \sigma_M^2) + (1 - \pi_M) \mathcal{N}(w; \mu_U, \sigma_U^2) \quad (24)$$

Fit via EM on the observed w (no labels needed). The optimal threshold is the intersection point of the two components. This removes the need for any labelled data for threshold selection.

8.1.2 Further exploration into hierarchical probabilistic linkage

Extend the approach for hierarchical probabilistic linkage mentioned in 5.5.

The hierarchical, two-stage probabilistic linkage engine introduced in Section 5.5 provides an operational voting mechanism by passing group-level aggregation scores as an input vector ($\mathbf{x} = [x_1, x_2, \dots, x_D]$) into a secondary Fellegi-Sunter loop. However, violating independence causes standard Expectation-Maximisation pipelines to "double-count" evidence, aggressively flattening the posterior calibration boundaries into artificial extremes (0.0 or 1.0).

Future work could consider replacing the independent product-space distribution with an explicit *Log-Linear Graphical Interaction Model* [Thibaudeau, 1993, Winkler, 1993].

Switching the parameter estimation loop should balance the cumulative advantage of probabilistic voting, while protecting calibration. While computational complexity is of consideration, given some small, fixed number of fields (aggregation functions), the shift in computation should be negligible in effect.

8.1.3 Active Learning & Label Minimisation

Recent literature Mozafari et al. [2025], Rough and Faingezicht [2024], Binette and Steorts [2022] suggests that as few as 30–50 labelled pairs can effectively calibrate the FELLEGI-SUNTER model and select thresholds. Future work:

- Implement uncertainty sampling (max entropy) or diversity sampling on the match weight space.
- Benchmark label efficiency: F_1 vs number of clerical labels across datasets.
- Integrate with SPLINK’s `label_studio` or custom active learning loop.

8.1.4 Multi-Source ($K > 2$) Global Resolution

Extend the global layer to K datasets:

- Star schema: one “reference” source, $K - 1$ others linked to it.
- Clique schema: all pairwise links, then transitive closure.
- Consensus aggregation: $\mathcal{A}(W^{(1)}, \dots, W^{(K)})$ where $W^{(k)}$ is local scores vs reference.
- Constraint: Transitive consistency (if $A=B$ and $B=C$ then $A=C$) — currently not enforced.

8.1.5 Distributed SPLINK on Spark for Scale

SPLINK supports Spark backend. Test on:

- `large` config (100K+ entities, high cardinality).
- Blocking rule parallelism.
- EM convergence in distributed setting.

8.1.6 Temporal Entity Resolution

Entities evolve: attributes change over time (name change, address move). Current model assumes static attributes. Approaches:

- Time-decay in m -probabilities (recent matches more reliable).
- State-space model: latent entity attributes evolve; observations are noisy emissions.
- Incremental EM: update parameters as new data arrives.

8.1.7 Semantic Augmentation (LLM Hybrid)

While this work scopes out LLM matching, a hybrid is compelling:

- Use LLM embeddings as additional comparison fields in SPLINK (treated as categorical/vector).
- LLM for blocking: generate blocking keys from semantic understanding.
- LLM for active learning: generate hard negative examples for training.

8.2 Conclusion

This report presented a hybrid entity resolution framework combining **local probabilistic matching** (FELLEGI-SUNTER via SPLINK) with **global aggregation** (scalar and vector-space).

Key findings:

1. The FELLEGI-SUNTER model with EM estimation, implemented in SPLINK, provides a principled and practical local matching layer under unsupervised constraints.
2. Blocking rule design with recall-bounding is the critical engineering lever; the “strictest rules” π initialisation heuristic is robust across configs.
3. Global aggregation is necessary when entities have multiple records per source. Scalar aggregations (mean, max, noisy-OR) have distinct regimes of validity; vector-space profile aggregation + profile-based SPLINK adds complexity but requires a profile-able structure in data.
4. At low cardinality, the local matching breaks down due to low discriminatory factors between pairs. The proposed profile-based SPLINK, over aggregating low-entropy aggregation scores, a promising direction.
5. On real-world data, the framework achieves competitive performance metrics without any labelled training data.

The framework is characterised by:

- **Explicit assumptions:** Conditional independence, stable within-dataset IDs, attribute-type scope.
- **Interpretable outputs:** Match weights decompose into per-field contributions.
- **Configurable trade-offs:** Blocking recall vs compute; aggregation method vs cardinality regime; threshold vs precision/recall.

- **Extensible architecture:** New comparison levels, aggregation functions, and layers compose cleanly.

The codebase (`generate_data.py`, `data_generator/`, `em/`, `aggregation/`) provides a reproducible foundation for further research on scalable, unsupervised entity resolution.

References

- Olivier Binette and Rebecca C. Steorts. (almost) all of entity resolution. *Science Advances*, 8(12):eabi8021, 2022. URL <https://science.org>.
- Arthur P. Dempster, Nan M. Laird, and Donald B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)*, 39(1):1–22, 1977.
- Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328):1183–1210, 1969.
- Robin Linacre, Sam Lindsay, et al. Splink: Probabilistic record linkage and deduplication at scale. Technical report, UK Ministry of Justice, 2024. Core mathematical ports derived from fastLink (Enamorado, Fifield, and Imai, 2019).
- Barzan Mozafari, Purnamrita Sarkar, Michael Franklin, Michael Jordan, and Samuel Madden. Active learning for entity resolution at scale. In *Proceedings of the VLDB Endowment*, 2025.
- Ethan Rough and Aaron Faingezicht. Roughly everything you need to know about entity resolution: Minimal pipelines for warehouse matching. *Technical Review and Guide*, 2024.
- Yves Thibaudeau. The discrimination power of dependency structures in record linkage. Research Report Series RR-93/01, US Bureau of the Census, Statistical Research Division, 1993.
- William E. Winkler. Methods for frequency-based matching in the Fellegi-Sunter model of record linkage. Statistical Research Division Report Series RR-89/02, U.S. Bureau of the Census, 1989.
- William E. Winkler. Improved decision rules in the Fellegi-Sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods, American Statistical Association*, pages 274–279, 1993.

A Summary of Entity Resolution Methodologies

This annex provides a structured overview of major ER approaches, their inference styles, and the problem structures they address. The choice of method is dictated by the constraints of the specific ER problem (data types, label availability, scale, attribute semantics).

Method Family	Inference Style	Problem Structure	Structure	Key Assumptions	Limitations
FELLEGI-SUNTER (FS)	EM / MLE (unsupervised)	Pairwise independent match status	classification; record pairs given	Conditional independence of fields; stationarity; comparison functions fixed	Independence violated in practice; requires good comparison functions; no transitive consistency

Method Family	Inference Style	Problem Structure	Structure	Key Assumptions	Limitations
Bayesian Networks / Graphical Models	MCMC / Variational Inference	Collective resolution; dependencies between pairs		Graph structure captures dependencies; prior knowledge encodable	Computationally heavy; requires structure learning or domain specification
Hierarchical Clustering	Agglomerative / Divisive	No labels; distance matrix given		Metric space; linkage criterion matches cluster shape	$\mathcal{O}(n^2)$ or $\mathcal{O}(n^3)$; sensitive to noise; no probabilistic output
Correlation Clustering	ILP / LP Relaxation / Approx	Partition into clusters; pairwise similarity		Objective: maximise agreements minus disagreements	NP-hard; approximations lose guarantees; no explicit match probabilities
Supervised Binary Classification	Gradient Boosting / NN / SVM	Labelled pairs; features from record pairs		IID pairs; sufficient labels; feature engineering captures signal	Needs labels; pairs not IID (transitivity); class imbalance severe
Learning to Rank	LambdaMART / ListNet	Rank candidates for each query record		Relevance grades available; listwise objective	Needs labelled lists; pointwise reduction loses list structure
Deep Metric Learning / Siamese Nets	Contrastive / Triplet Loss	Learn embedding where matches are close		Large labelled data; architecture captures semantics	Black box; needs massive data; no direct probability
LLM Semantic Matching	Prompting / Fine-tuning / Embeddings	Text-heavy records; semantic equivalence		LLM understands domain; prompts generalise	Costly; non-deterministic; hard to calibrate; no structure exploited
Graph / Behavioural ER	Message Passing / GNN / Random Walk	Network / transactional behaviour identity	trans data; defines	Entity behaviour is distinctive; graph structure informative	Requires graph construction; different feature space (behavioural vs attribute)
Canonicalisation / Standardisation + Exact Match	Rule-based / Deterministic	Cleanable attributes; canonical forms exist		Rules are complete and correct	Brittle; manual effort; fails on fuzzy/noisy data
Local-to-Global (This Work)	FS + Aggregation (unsupervised)	Stable within-dataset IDs; multi-record entities		Within-dataset ID stability; attribute types (text/num/-date); conditional independence	Fails at low cardinality (indep breaks); 2 datasets only; static entities

Decision Guidance:

- *No labels, attribute data, stable IDs $\rightarrow$ FS (this work)*
- *No labels, attribute data, NO stable IDs $\rightarrow$ Clustering / Graphical Models*
- *Labels available, attribute data $\rightarrow$ Supervised / LTR / Deep Metric*
- *Text-heavy, semantic variation $\rightarrow$ LLM Embeddings + FS/Supervised*

- *Behavioural / Network data* $\rightarrow$ *Graph ER / GNN*
- *Multi-source (> 2), transitive consistency needed* $\rightarrow$ *Collective / Graphical*
- *Temporal evolution* $\rightarrow$ *State-space / Incremental*

B Mathematical Derivations

C Fellegi-Sunter Model Derivation

C.1 From Hypothesis Test to Match Weight

For a record pair (r_1, r_2) , we observe comparison vector $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_K)$. The likelihood ratio is:

$$\Lambda(\boldsymbol{\gamma}) = \frac{\mathbb{P}(\boldsymbol{\gamma} \mid M)}{\mathbb{P}(\boldsymbol{\gamma} \mid U)} \quad (25)$$

Under conditional independence:

$$\Lambda(\boldsymbol{\gamma}) = \prod_{k=1}^K \frac{m_k(\gamma_k)}{u_k(\gamma_k)} \quad (26)$$

The match weight (log-likelihood ratio, base 2):

$$w(\boldsymbol{\gamma}) = \log_2 \Lambda(\boldsymbol{\gamma}) = \sum_{k=1}^K \log_2 \frac{m_k(\gamma_k)}{u_k(\gamma_k)} \quad (27)$$

C.2 Posterior Match Probability

Let $\pi = \mathbb{P}(M)$ be the prior match probability in the comparison space. By Bayes' theorem:

$$\mathbb{P}(M \mid \boldsymbol{\gamma}) = \frac{\mathbb{P}(\boldsymbol{\gamma} \mid M)\pi}{\mathbb{P}(\boldsymbol{\gamma} \mid M)\pi + \mathbb{P}(\boldsymbol{\gamma} \mid U)(1 - \pi)} \quad (28)$$

$$= \frac{\Lambda(\boldsymbol{\gamma})\pi}{\Lambda(\boldsymbol{\gamma})\pi + (1 - \pi)} \quad (29)$$

$$= \frac{2^{w(\boldsymbol{\gamma})}\pi}{2^{w(\boldsymbol{\gamma})}\pi + (1 - \pi)} \quad (30)$$

Solving for w :

$$w = \log_2 \left(\frac{\mathbb{P}(M \mid \boldsymbol{\gamma})}{1 - \mathbb{P}(M \mid \boldsymbol{\gamma})} \cdot \frac{1 - \pi}{\pi} \right) \quad (31)$$

C.3 EM Algorithm Derivation

The complete-data log-likelihood for N pairs with latent match indicators $Z_i \in \{0, 1\}$:

$$\ell_c(\boldsymbol{\theta}) = \sum_{i=1}^N [Z_i \log \mathbb{P}(\boldsymbol{\gamma}_i \mid M) + (1 - Z_i) \log \mathbb{P}(\boldsymbol{\gamma}_i \mid U)] \quad (32)$$

with $\boldsymbol{\theta} = \{\pi, \{m_k(\gamma)\}, \{u_k(\gamma)\}\}$.

E-step: Compute $\tau_i = \mathbb{E}[Z_i \mid \boldsymbol{\gamma}_i, \boldsymbol{\theta}^{(t)}] = \mathbb{P}(M \mid \boldsymbol{\gamma}_i, \boldsymbol{\theta}^{(t)})$.

M-step: Maximise $\mathbb{E}[\ell_c \mid \gamma, \theta^{(t)}]$ w.r.t θ :

$$\pi^{(t+1)} = \frac{1}{N} \sum_{i=1}^N \tau_i \quad (33)$$

$$m_k^{(t+1)}(\gamma) = \frac{\sum_{i=1}^N \tau_i \mathbb{I}[\gamma_{ik} = \gamma]}{\sum_{i=1}^N \tau_i} \quad (34)$$

$$u_k^{(t+1)}(\gamma) = \frac{\sum_{i=1}^N (1 - \tau_i) \mathbb{I}[\gamma_{ik} = \gamma]}{\sum_{i=1}^N (1 - \tau_i)} \quad (35)$$

C.4 Blocked EM

When blocking rule B is applied, pairs outside B are not compared. The prior in the blocked space is $\pi_B = \frac{\mathbb{E}[\text{matches in } B]}{|B|}$. SPLINK estimates π_B via user-provided target recall R and strict blocking rule:

$$\hat{\pi}_B = \frac{R \cdot (\text{total true matches})}{|B|} \quad (36)$$

Total true matches $\approx \pi \cdot |A| \cdot |B|$ (cartesian), but π is unknown. SPLINK uses the strict rule (high precision) to estimate matches in strict rule, then scales by R .

D Aggregation Function Properties

Let $P \in [0, 1]^{m \times n}$ be the matrix of local pairwise match probabilities for global entity pair (e_A, e_B) .

Theorem D.1 (Noisy-OR Monotonicity). $s_{\text{noisy-or}}(P) = 1 - \prod_{i,j} (1 - p_{ij})$ is monotonically increasing in each p_{ij} and in m, n (adding pairs never decreases the score).

Proof. $\frac{\partial s}{\partial p_{ij}} = \prod_{(k,l) \neq (i,j)} (1 - p_{kl}) \geq 0$. Adding a new pair $(m+1, n+1)$ multiplies by $(1 - p_{\text{new}}) \leq 1$, so s increases or stays same. $\square$

Theorem D.2 (Max Aggregation Invariance). $s_{\text{max}}(P)$ depends only on the maximum entry. Adding dominated pairs has no effect.

Proof. Trivial from definition of maximum. $\square$

Theorem D.3 (Normalised Mean Cardinality Bias). $s_{\text{norm}}(P) = \frac{1}{mn} \sum p_{ij}$. If P has rank-1 structure $p_{ij} = a_i b_j$ (e.g., independent match signals per record), then $s_{\text{norm}} = (\frac{1}{m} \sum a_i)(\frac{1}{n} \sum b_j)$ — depends only on means, not on m, n individually.

E Set Similarity Metrics

E.1 Monge-Elkan

$\text{ME}(L, R) = \frac{1}{|L|} \sum_{x \in L} \max_{y \in R} \text{sim}(x, y)$. Asymmetric: $\text{ME}(L, R) \neq \text{ME}(R, L)$ in general.

E.2 Soft Jaccard

$S_{L \rightarrow R} = \sum_{x \in L} \max_{y \in R} \text{JW}(x, y)$, $S_{R \rightarrow L} = \sum_{y \in R} \max_{x \in L} \text{JW}(x, y)$. $\text{SJ}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{2|L \cup R|}$. Symmetric, range $[0, 1]$. When $|L| = |R| = 1$, $\text{SJ} = \text{JW}(x, y)$.

E.3 Magnitude-Weighted Cosine

For vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}_{\geq 0}^d$: $\text{MagCosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\max(\|\mathbf{u}\|^2, \|\mathbf{v}\|^2)}$. Downweights matches where one vector has much larger magnitude (catalog size imbalance).

F Cardinality Breakdown Analysis

Let e_A, e_B be true matching global entities with m and n records. True match pairs: mn . Non-match pairs involving these records: $m(N_B - n) + n(N_A - m) + (N_A - m)(N_B - n)$. Ratio of true to false local pairs for this entity: $\frac{mn}{mN_B + nN_A - mn}$. When m, n are small relative to N_A, N_B , this ratio is tiny — the signal is drowned in noise. When m, n are large, the ratio grows, but the comparison space explodes.

The independence violation: Covariance of two local comparisons sharing a record: $\text{Cov}(\gamma(r_{A,1}, r_{B,1}), \gamma(r_{A,1}, r_{B,2})) = \mathbb{E}[\gamma_1 \gamma_2] - \mathbb{E}[\gamma_1] \mathbb{E}[\gamma_2]$. Since both depend on the true value of $r_{A,1}$, this is positive. The effective number of independent comparisons is less than mn . Layer-2 addresses this by using a correlation-adjusted likelihood.

G Data Generation Implementation

This annex contains the core data generation pipeline for reproducibility.

H Main Entry Point: generate_data.py

```
1 import os
2 import yaml
3 import pandas as pd
4 import numpy as np
5 from data_generator.entitymanager import EntitySpaceManager
6 from data_generator.pipeline import DataPipelineOrchestrator
7
8 SEED = 42
9 rng = np.random.default_rng(SEED)
10 SIZE = "smaller" # Config key: smaller_config.yaml
11
12 def assign_latent_event_ids(df, id_prefix, start_id=0):
13     """Assign explicit latent event IDs to a dataframe."""
14     n_rows = len(df)
15     if n_rows == 0:
16         return df, start_id
17     latent_ids = [f"{id_prefix}_{start_id + i}" for i in range(n_rows)]
18     df = df.copy()
19     df["latent_event_id"] = latent_ids
20     return df, start_id + n_rows
21
22 def main():
23     config_path = os.path.join("data_generator", f"{SIZE}_config.yaml")
24     if not os.path.exists(config_path):
25         raise FileNotFoundError(f"Configuration file not found at: {config_path}")
26
27     print(f"Loading system configuration from {config_path}...")
28     with open(config_path, "r", encoding="utf-8") as f:
29         config = yaml.safe_load(f)
30
31     sim_config = config["simulation"]
32     dataset_size = sim_config["dataset_size"]
33     overlap_percentage = sim_config["overlap_percentage"]
```

```

34
35 overlap_count = int(dataset_size * overlap_percentage)
36 unique_count_per_source = dataset_size - overlap_count
37
38 print(f"Generating {overlap_count} overlapping entities and {unique_count_per_source}
39 unique entities per source...")
40
41 # 1. Build the true underlying entity identity pool
42 space_manager = EntitySpaceManager(
43     dataset_size=dataset_size, overlap_rate=overlap_percentage, rng=rng
44 )
45
46 # 2. Extract the non-corresponding identity mapping tables
47 print("Generating non-corresponding identity maps...")
48 mapping_a, mapping_b = space_manager.generate_source_mappings()
49
50 # 3. Split mappings into unique and overlapping portions
51 mapping_a_overlap = mapping_a.iloc[:overlap_count].copy()
52 mapping_a_unique = mapping_a.iloc[overlap_count:].copy()
53 mapping_b_overlap = mapping_b.iloc[:overlap_count].copy()
54 mapping_b_unique = mapping_b.iloc[overlap_count:].copy()
55
56 print(f"Mapping A: {len(mapping_a_overlap)} overlapping + {len(mapping_a_unique)} unique")
57 print(f"Mapping B: {len(mapping_b_overlap)} overlapping + {len(mapping_b_unique)} unique")
58
59 # 4. Instantiate the orchestration pipeline
60 print("Executing vectorised multi-row data orchestration pipeline...")
61 orchestrator = DataPipelineOrchestrator(config=config, rng=rng)
62
63 # 5. Generate the three foundational datasets
64 print("Generating overlapping dataset base...")
65 overlapping_base = orchestrator.generate_dataset(
66     mapping_a_overlap,
67     "source_a_rows_per_entity",
68     overlap_count,
69     0
70 )
71
72 print("Generating dataset A unique portion...")
73 dataset_a_unique = orchestrator.generate_dataset(
74     mapping_a_unique,
75     "source_a_rows_per_entity",
76     0,
77     unique_count_per_source
78 )
79
80 print("Generating dataset B unique portion...")
81 dataset_b_unique = orchestrator.generate_dataset(
82     mapping_b_unique,
83     "source_b_rows_per_entity",
84     0,
85     unique_count_per_source
86 )
87
88 # 6. Assign explicit latent event ID labels
89 print("Assigning latent event ID labels...")
90 next_id = 0
91 overlapping_base, next_id = assign_latent_event_ids(overlapping_base, "OVER", next_id)
92 dataset_a_unique, next_id = assign_latent_event_ids(dataset_a_unique, "UNI_A", next_id)
93 dataset_b_unique, next_id = assign_latent_event_ids(dataset_b_unique, "UNIQ_B", next_id)
94
95 # 7. Create the two noisy versions from overlapping base
96 print("Creating noisy version A from overlapping base...")

```

```

96 dataset_a_overlap = apply_noise_to_base(overlapping_base.copy(), config, rng, source='a')
97
98 print("Creating noisy version B from overlapping base (with dropout)...")
99 dataset_b_overlap = apply_noise_to_base(overlapping_base.copy(), config, rng, source='b')
100 dataset_b_overlap = apply_dropout(dataset_b_overlap, config, rng)
101
102 # 8. Combine to create final datasets
103 print("Combining datasets...")
104 dataset_a = pd.concat([dataset_a_overlap, dataset_a_unique], ignore_index=True)
105 dataset_b = pd.concat([dataset_b_overlap, dataset_b_unique], ignore_index=True)
106
107 # 9. Optionally sort by datetime column
108 dataset_a = sort_by_datetime_if_exists(dataset_a, config)
109 dataset_b = sort_by_datetime_if_exists(dataset_b, config)
110
111 # 10. Save out files
112 output_dir = "output_data"
113 os.makedirs(output_dir, exist_ok=True)
114
115 print(f"Persisting files to disk under folder: '{output_dir}/'...")
116 dataset_a.to_parquet(os.path.join(output_dir, f"{SIZE}_dataset_a.parquet"), index=False)
117 dataset_b.to_parquet(os.path.join(output_dir, f"{SIZE}_dataset_b.parquet"), index=False)
118
119 print("\nGeneration pipeline run complete. Summary profile metrics:")
120 print(f"-> Dataset A: {len(dataset_a):,} rows generated.")
121 print(f"  - Overlapping portion: {len(dataset_a_overlap):,} rows")
122 print(f"  - Unique portion: {len(dataset_a_unique):,} rows")
123 print(f"-> Dataset B: {len(dataset_b):,} rows generated.")
124 print(f"  - Overlapping portion: {len(dataset_b_overlap):,} rows")
125 print(f"  - Unique portion: {len(dataset_b_unique):,} rows")
126 print("Data structures are successfully saved and ready for the Entity Resolution engine.")
127
128 def apply_noise_to_base(base_df, config, rng, source='a'):
129     """Apply feature-specific noise to overlapping base dataset. Preserves ID columns."""
130     preserve_cols = ['global_entity_id', 'local_source_id', 'latent_event_id']
131     cols_to_noise = [col for col in base_df.columns if col not in preserve_cols]
132
133     df_noised = base_df.copy()
134
135     for col in cols_to_noise:
136         feat_config = next((f for f in config["features"] if f["name"] == col), None)
137         if feat_config:
138             f_type = feat_config["type"]
139             from data_generator.generator import DataGeneratorFactory
140             generator = DataGeneratorFactory.get_generator(f_type, rng=rng)
141             df_noised[col] = generator.apply_noise(base_df[col].values, feat_config)
142
143     return df_noised
144
145 def apply_dropout(dataset_b, config, rng):
146     """Apply asymmetric dropout to overlapping rows in dataset B only."""
147     sim_config = config["simulation"]
148     dropout_frac = sim_config.get("overlap_dropout_frac", 0.0)
149
150     if dropout_frac <= 0.0:
151         return dataset_b
152
153     n_rows = len(dataset_b)
154     n_to_drop = int(n_rows * dropout_frac)
155
156     if n_to_drop > 0:
157         drop_indices = rng.choice(dataset_b.index, size=n_to_drop, replace=False)
158         dataset_b = dataset_b.drop(index=drop_indices).reset_index(drop=True)

```

```

159
160     return dataset_b
161
162 def sort_by_datetime_if_exists(df, config):
163     """Sort dataset by datetime column if one exists."""
164     dt_col = next((f["name"] for f in config["features"] if f["type"] == "datetime"), None)
165     if dt_col and dt_col in df.columns:
166         df = df.sort_values(by=dt_col).reset_index(drop=True)
167     return df
168
169 if __name__ == "__main__":
170     main()

```

Listing 1: generate_data.py – Core Pipeline

I Configuration: data_generator/config.yaml

```

1 simulation:
2   dataset_size: 100                # Total global entities (NOT total rows)
3   overlap_percentage: 0.10         # 10% true aggregate matches
4   source_a_rows_per_entity: 100    # Simulates multiple transactions per person
5   source_b_rows_per_entity: 120    # Asymmetric cardinality
6   overlap_dropout_frac: 0.0        # Optional: drop overlapping rows from B
7
8 features:                          # Features used to generate the data
9   - name: "categorical"
10     type: "categorical"
11     num_classes: 25
12     missing_rate: 0.05
13
14   - name: "numerical_noisy"
15     type: "numerical"
16     noise_type: "gaussian"
17     noise_sigma: 2.5
18     missing_rate: 0.05
19
20   - name: "datetime_noisy"
21     type: "datetime"
22     start_date: "2026-01-01"
23     end_date: "2026-06-01"
24     noise_type: "gaussian"
25     noise_sigma_hours: 6.0
26     missing_rate: 0.00
27
28 available_features:                # Pool of available feature definitions
29   - name: "categorical"
30     type: "categorical"
31     num_classes: 25
32     missing_rate: 0.05
33
34   - name: "numerical_noisy"
35     type: "numerical"
36     noise_type: "gaussian"
37     noise_sigma: 2.5
38     missing_rate: 0.0
39
40   - name: "numerical_clean"
41     type: "numerical"
42     noise_type: "none"
43     missing_rate: 0.05
44
45   - name: "datetime_noisy"

```

```

46     type: "datetime"
47     start_date: "2026-01-01"
48     end_date: "2026-06-01"
49     noise_type: "gaussian"
50     noise_sigma_hours: 2.0
51     missing_rate: 0.02
52
53 - name: "datetime_clean"
54   type: "datetime"
55   start_date: "2026-01-01"
56   end_date: "2026-06-01"
57   noise_type: "none"
58   missing_rate: 0.0

```

Listing 2: config.yaml – Simulation & Feature Configuration

J Core Modules

J.1 data_generator/entitymanager.py

Manages the global entity pool and source mappings.

```

1 # data_generator/entitymanager.py (simplified interface)
2 class EntitySpaceManager:
3     def __init__(self, dataset_size, overlap_rate, rng):
4         self.dataset_size = dataset_size
5         self.overlap_rate = overlap_rate
6         self.rng = rng
7         self.global_entities = self._create_entity_pool()
8
9     def _create_entity_pool(self):
10        # Create pool of global entity IDs with true attribute values
11        pass
12
13    def generate_source_mappings(self):
14        # Split entities into Source A and Source B mappings
15        # overlap_rate fraction appears in both
16        # Returns: mapping_a, mapping_b DataFrames with global_entity_id, latent attributes
17        pass

```

Listing 3: EntitySpaceManager – Identity Pool

J.2 data_generator/pipeline.py

Orchestrates record expansion and feature generation.

```

1 # data_generator/pipeline.py (simplified interface)
2 class DataPipelineOrchestrator:
3     def __init__(self, config, rng):
4         self.config = config
5         self.rng = rng
6         self.generators = self._build_generators()
7
8     def _build_generators(self):
9        # Instantiate feature generators from config
10        pass
11
12    def generate_dataset(self, mapping, rows_per_entity_col, overlap_count, unique_count):
13        # Expand each entity into multiple records (rows)
14        # Apply latent attribute values
15        # Returns: DataFrame with global_entity_id, local_source_id, feature columns

```

Listing 4: DataPipelineOrchestrator – Record Generation

J.3 data_generator/generator.py

Feature-specific generation and noise application.

```

1 # data_generator/generator.py (simplified interface)
2 class DataGeneratorFactory:
3     @staticmethod
4     def get_generator(feature_type, rng):
5         if feature_type == "categorical":
6             return CategoricalGenerator(rng)
7         elif feature_type == "numerical":
8             return NumericalGenerator(rng)
9         elif feature_type == "datetime":
10            return DatetimeGenerator(rng)
11
12 class CategoricalGenerator:
13     def apply_noise(self, values, feat_config):
14         # With missing_rate -> NaN
15         # With prob (1-missing) * (1/num_classes) -> random other class
16         # Else -> keep true value
17         pass
18
19 class NumericalGenerator:
20     def apply_noise(self, values, feat_config):
21         # values + N(0, sigma^2); missing_rate -> NaN
22         pass
23
24 class DatetimeGenerator:
25     def apply_noise(self, values, feat_config):
26         # values + N(0, sigma_hours^2); missing_rate -> NaT
27         pass

```

Listing 5: DataGeneratorFactory + Noise Models

K Unified Soft String-Matching Metrics Framework

L Definitions and Notation

Let L and R be two token lists representing the left and right records, respectively. Let $|L|$ and $|R|$ denote the total number of token slots in each list. Let $JW(x, y)$ represent the Jaro-Winkler similarity score between token x and token y .

We define the directional **Best-Match Sums** from left-to-right ($S_{L \rightarrow R}$) and right-to-left ($S_{R \rightarrow L}$) as follows:

$$S_{L \rightarrow R} = \sum_{x \in L} \max_{y \in R} JW(x, y) \quad (37)$$

$$S_{R \rightarrow L} = \sum_{y \in R} \max_{x \in L} JW(x, y) \quad (38)$$

M The Four Soft Metrics

M.1 Monge-Elkan Similarity (Asymmetric)

Measures the average of the best-matching scores from the query set (L) to the reference set (R):

$$\text{MongeElkan}(L, R) = \frac{S_{L \rightarrow R}}{|L|} \quad (39)$$

M.2 Symmetric Soft Jaccard Similarity

Normalizes the combined mutual best-match scores by the total size of the distinct token union:

$$\text{SoftJaccard}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{2 \cdot |L \cup R|_{\text{distinct}}} \quad (40)$$

M.3 Symmetric Soft Dice Coefficient

Normalizes the combined mutual best-match scores by the total raw token slots available across both lists:

$$\text{SoftDice}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{|L| + |R|} \quad (41)$$

M.4 Soft Simpson Coefficient (Overlap Metric)

Normalizes the average mutual best-match score by the size of the smaller token list, isolating it from catalog size imbalances:

$$\text{SoftSimpson}(L, R) = \frac{S_{L \rightarrow R} + S_{R \rightarrow L}}{2 \cdot \min(|L|, |R|)} \quad (42)$$

N Unified Binned Comparison Logic

For any given metric score S , the assignment to a Splink comparison level is generalized using the following case structure:

$$\text{ComparisonLevel}(S) = \begin{cases} \text{Null / Empty} & \text{if } L \text{ is Null, } R \text{ is Null, } |L| = 0, \text{ or } |R| = 0 \\ \text{High Similarity} & \text{if } S \geq \tau_{\text{high}} \\ \text{Medium Similarity} & \text{if } S \geq \tau_{\text{med}} \\ \text{Low Similarity} & \text{if } S \geq \tau_{\text{low}} \\ \text{Else} & \text{otherwise} \end{cases} \quad (43)$$

where the operational mapping thresholds are defined as:

O DuckDB SQL Implementations

O.1 Soft Jaccard

```
1 -- Sum of best matches from L to R
2 WITH max_sims_l AS (
3     SELECT list_aggregate(
4         list_transform(l.tokens, x ->
```

Table 4: Operational Level Thresholds

Metric	τ_{high}	τ_{med}	τ_{low}
Monge-Elkan	0.85	0.70	0.55
Soft Jaccard	0.85	0.65	0.40
Soft Dice	0.85	0.65	0.40
Soft Simpson	0.85	0.65	0.40

```

5         list_max(list_transform(r.tokens, y -> jaro_winkler_similarity(x, y)))
6     ), 'sum'
7 ) AS sum_l
8 ),
9 -- Sum of best matches from R to L
10 max_sims_r AS (
11     SELECT list_aggregate(
12         list_transform(r.tokens, y ->
13             list_max(list_transform(l.tokens, x -> jaro_winkler_similarity(x, y)))
14         ), 'sum'
15     ) AS sum_r
16 ),
17 -- Union size
18 union_size AS (
19     SELECT len(list_distinct(list_concat(l.tokens, r.tokens))) AS u_size
20 )
21 SELECT (sum_l + sum_r) / (2.0 * u_size) AS soft_jaccard
22 FROM max_sims_l, max_sims_r, union_size;

```

Listing 6: Soft Jaccard SQL

O.2 Monge-Elkan

```

1 WITH max_sims AS (
2     SELECT list_aggregate(
3         list_transform(l.tokens, x ->
4             list_max(list_transform(r.tokens, y -> jaro_winkler_similarity(x, y)))
5         ), 'sum'
6     ) AS sum_l
7 ),
8 len_l AS (
9     SELECT len(l.tokens) AS n
10 )
11 SELECT sum_l / n AS monge_elkan
12 FROM max_sims, len_l;

```

Listing 7: Monge-Elkan SQL

O.3 Soft Dice

```

1 WITH max_sims_l AS (...), max_sims_r AS (...)
2 SELECT (sum_l + sum_r) / (len(l.tokens) + len(r.tokens)) AS soft_dice
3 FROM max_sims_l, max_sims_r;

```

Listing 8: Soft Dice SQL

O.4 Soft Simpson

```

1 WITH max_sims_l AS (...), max_sims_r AS (...)
2 SELECT (sum_l + sum_r) / (2.0 * least(len(l.tokens), len(r.tokens))) AS soft_simpson

```



```
3 FROM max_sims_l, max_sims_r;
```

Listing 9: Soft Simpson SQL

P Properties Summary

Table 5: Metric Properties Comparison

Property	Monge-Elkan	Soft Jaccard	Soft Dice	Soft Simpson
Symmetric	No	Yes	Yes	Yes
Range	$[0, 1]$	$[0, 1]$	$[0, 1]$	$[0, 1]$
Reduces to JW when $ L = R = 1$	Yes	Yes	Yes	Yes
Normalisation	By query size	By union	By total tokens	By smaller set
Catalog-size robust	No	Partial	Partial	Yes
Asymmetric (L vs R matters)	Yes	No	No	No

Q Usage Recommendations

- **Monge-Elkan:** When L is a query (e.g., user input) and R is a reference catalog. Asymmetric by design.
- **Soft Jaccard:** General-purpose symmetric similarity; normalised by union so penalises extra tokens on either side.
- **Soft Dice:** Similar to Soft Jaccard but normalised by arithmetic mean of sizes; slightly higher scores for imbalanced sets.
- **Soft Simpson:** When catalog size imbalance is the main confounder (e.g., one entity has 50 title tokens, other has 1). Isolates overlap quality from volume.

R Artist Matching Ground Truth and Precision/Recall Guarantees

R.1 Subset Precision is a Lower Bound on True Precision

Let G_{true} be the total hidden ground truth of true matches, G_{sub} be the trusted subset where $G_{sub} \subseteq G_{true}$, and M be the set of matches predicted by the model.

The true precision and the subset precision are defined as:

$$\text{Precision}_{true} = \frac{|M \cap G_{true}|}{|M|}$$

$$\text{Precision}_{sub} = \frac{|M \cap G_{sub}|}{|M|}$$

Since the trusted subset is a subset of the total true matches:

$$G_{sub} \subseteq G_{true}$$

By the monotonicity of set intersections, intersecting both sides with the model predictions M yields:

$$(M \cap G_{sub}) \subseteq (M \cap G_{true})$$

Taking the cardinality (size) of both sets preserves the inequality:

$$|M \cap G_{sub}| \leq |M \cap G_{true}|$$

Dividing both sides by the strictly positive total number of model predictions $|M|$ gives:

$$\frac{|M \cap G_{sub}|}{|M|} \leq \frac{|M \cap G_{true}|}{|M|}$$

Substituting the original definitions completes the proof:

$$\text{Precision}_{sub} \leq \text{Precision}_{true} \quad \blacksquare$$

R.2 Artist Matching Prior and Recall Estimation under Clustered Noise

Let $\mathcal{M}$ be the global set of true matches, $\mathcal{S}_{\text{clean}}$ be the exact artist match subset, and $\mathcal{A}$ be the model predictions. We isolate two distinct fractional components within our evaluation framework:

$$p = \frac{|\mathcal{M} \cap \mathcal{S}_{\text{clean}}|}{|\mathcal{S}_{\text{clean}}|} \quad (\text{The True Match Prior} / \text{Prevalence Proportion}) \quad (44)$$

$$q = \frac{|\mathcal{A} \cap \mathcal{M} \cap \mathcal{S}_{\text{clean}}|}{|\mathcal{M} \cap \mathcal{S}_{\text{clean}}|} \quad (\text{The Subset Capturing Success Rate} / \text{Estimated Recall } \hat{R}_{\text{eval}}) \quad (45)$$

R.2.1 Preservation of the True Match Prior (p)

Because the artificial noising process on the artist column is uniform and random, the clean-artist property $\mathcal{S}_{\text{clean}}$ is statistically independent of the true latent matching state $\mathcal{M}$:

$$\mathcal{S}_{\text{clean}} \perp\!\!\!\perp \mathcal{M} \implies \frac{|\mathcal{S}_{\text{clean}} \cap \mathcal{M}|}{|\mathcal{M}|} = \frac{|\mathcal{S}_{\text{clean}}|}{|\mathcal{P}|} \quad (46)$$

By re-arranging the terms, we formally prove that your baseline proportion p matches global reality:

$$p = \frac{|\mathcal{M} \cap \mathcal{S}_{\text{clean}}|}{|\mathcal{S}_{\text{clean}}|} = \frac{|\mathcal{M}|}{|\mathcal{P}|} = p_{\text{global}} \quad (47)$$

R.2.2 Clustered Noise and Captured Predictive Inflation (q)

Let $\mathcal{Q} \in \{0, 1\}$ be a latent data quality indicator, where $\mathcal{Q} = 1$ represents uncorrupted records.

In real-world data streams, data quality naturally travels in systemic clusters across columns:

$$\text{Quality}(\text{Song Features}) \not\perp\!\!\!\perp \text{Quality}(\text{Artist Name}) \quad (48)$$

Therefore, pulling record pairs into $\mathcal{S}_{\text{clean}}$ drastically forces the latent quality state $\mathcal{Q} \rightarrow 1$:

$$\frac{|\mathcal{S}_{\text{clean}} \cap \{\mathcal{Q} = 1\}|}{|\mathcal{S}_{\text{clean}}|} \gg \frac{|\{\mathcal{Q} = 1\}|}{|\mathcal{P}|} \quad (49)$$

Let $\mathcal{X}$ represent the song features used by your model. The model's prediction success depends on $\mathcal{Q}$:

$$\left(\frac{|\mathcal{A} \cap \mathcal{M} \cap \{\mathcal{Q} = 1\}|}{|\mathcal{M} \cap \{\mathcal{Q} = 1\}|} \right) \gg \left(\frac{|\mathcal{A} \cap \mathcal{M} \cap \{\mathcal{Q} = 0\}|}{|\mathcal{M} \cap \{\mathcal{Q} = 0\}|} \right) \quad (50)$$

Expanding your subset success rate q over the hidden data quality distribution reveals the inflation:

$$q = \underbrace{\left(\frac{|\mathcal{A} \cap \mathcal{M} \cap \mathcal{S}_{\text{clean}} \cap \{\mathcal{Q} = 1\}|}{|\mathcal{M} \cap \mathcal{S}_{\text{clean}} \cap \{\mathcal{Q} = 1\}|} \right)}_{\text{Artificially maximized success rate}} \cdot \left(\frac{|\mathcal{M} \cap \mathcal{S}_{\text{clean}} \cap \{\mathcal{Q} = 1\}|}{|\mathcal{M} \cap \mathcal{S}_{\text{clean}}|} \right) + \epsilon \quad (51)$$

Because the weight of the pristine $\{\mathcal{Q} = 1\}$ records is highly concentrated inside your evaluation set:

$$q \gg \frac{|\mathcal{A} \cap \mathcal{M}|}{|\mathcal{M}|} = R_{\text{global}} \quad (52)$$

Conclusion 2: The model captures a significantly higher proportion of positives inside this subset than globally.